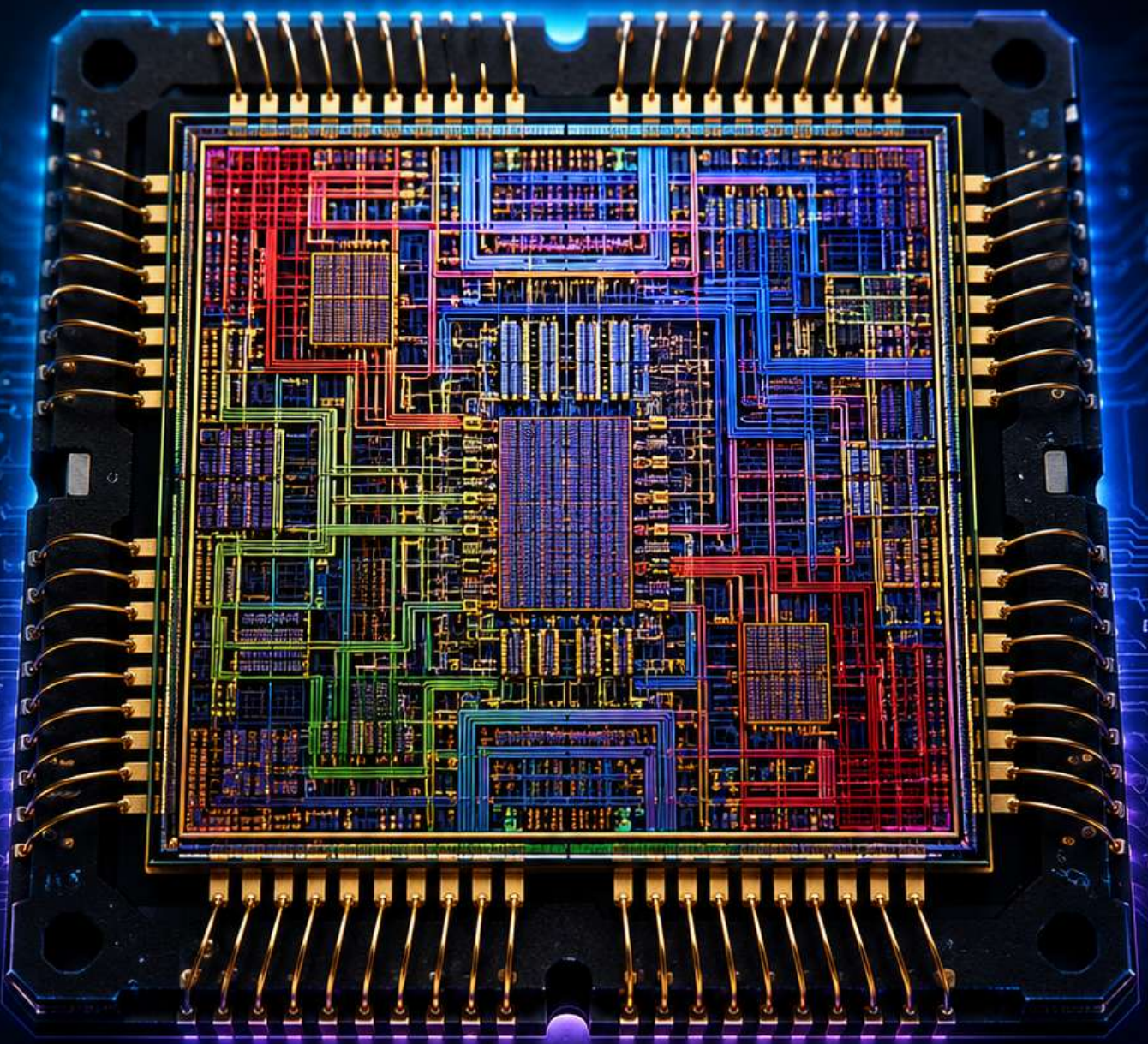


PHYSICAL DESIGN FLOW

RTL to GDSII — A Complete Script-by-Script Reference



Banothu Bapuji

About the Book

Physical Design Flow: RTL to GDSII — A Complete Script-by-Script Reference is a hands-on technical guide that walks readers through a real, working ASIC physical design flow, built entirely on Synopsys Design Compiler and IC Compiler II targeting a 14nm reference technology node. Using an 8-bit FIFO chip as a running example, the book reproduces every script in the flow exactly as executed — from RTL source and logic synthesis through data preparation, multi-corner/multi-mode setup, floorplanning, pad frame construction, power planning, placement, clock tree synthesis, and detailed routing.

Rather than offering abstract theory, each chapter presents the complete, unedited script followed by a rigorous command-by-command explanation, connecting every Tcl command to the underlying physical design concept it implements. Along the way, the book captures the practical realities of real flow development — engineering decisions, databook-verified library settings, known issues, and design trade-offs — offering readers an authentic view of how a production-style RTL-to-GDSII flow is actually built and debugged.

Written for graduate students, VLSI engineers, and physical design practitioners, this book bridges the gap between conceptual PD coursework and the script-level fluency demanded in industry, making it a practical companion for anyone learning to take a digital design from RTL all the way to a manufacturable GDSII layout.

About the Author

Banothu Bapuji is a VLSI design engineer and researcher with expertise in digital physical design (RTL-to-GDSII) and analog/mixed-signal integrated circuit design. His work focuses on developing high-performance, power-efficient semiconductor systems by combining practical silicon implementation with advanced design automation methodologies.

He has successfully implemented a complete image-dehazing ASIC, gaining hands-on experience across every stage of the RTL-to-GDSII flow, including floorplanning, power planning, placement, clock tree synthesis, routing, timing optimization, physical verification, and signoff using industry-standard EDA tools. The practical knowledge gained through this work forms the foundation of the methodologies presented throughout this book.

In addition to digital physical design, he has designed and verified analog building blocks such as CMOS operational amplifiers, low-dropout (LDO) voltage regulators, and bandgap reference circuits, with research extending to cryogenic electronics and high-reliability integrated circuit applications.

Banothu Bapuji has authored more than a dozen peer-reviewed research articles published in reputed international journals and conferences, holds a filed patent, and has presented his research at international technical forums. His research interests include physical design automation, AI-assisted electronic design automation (EDA), reinforcement learning for VLSI optimization, hardware accelerators for computer vision, approximate computing, and next-generation semiconductor architectures.

Passionate about transforming innovative ideas into manufacturable silicon, he is committed to advancing intelligent chip design by integrating artificial intelligence with modern physical design methodologies, enabling the development of efficient, scalable, and reliable integrated circuits for future computing systems.

A Note on the Figures

The scripts and command-by-command explanations presented throughout this book are based on real, working RTL-to-GDSII flow scripts, reflecting an authentic implementation rather than a simplified example. Due to certain constraints in the preparation of this manuscript, the corresponding tool-generated screenshots and layout images could not be captured directly from the EDA environment at each stage.

In their place, AI-generated illustrative images have been used to visually represent the expected tool outputs, console logs, and layout views described in the text. These images are intended purely as visual aids to support understanding and do not represent actual captured tool output.

Readers reproducing this flow in their own environment are encouraged to run the scripts and capture genuine tool-generated outputs at each stage, as the scripts themselves have been verified to execute correctly and produce the described results.

Banothu Bapuji
VLSI PD RTL-TO GDS USING SYNOPSYS TOOLS

Physical Design Flow

RTL to GDSII

A Complete Script-by-Script Reference

8-bit FIFO Chip (FIFO_ChipTop)

Synopsys Design Compiler + IC Compiler II | SAED14nm_EDK_03_2025 (HVT)

Covers:

*common_setup · RTL Sources · dc_script · data_preparation · mcm floorplan
· pad_frame · powerplan · placement · cts · route*

Banothu Bapuji
VLSI PD RTL-TO GDS USING SYNOPSYS TOOLS

Table of Contents

Table of Contents	2
Chapter 1 — The Complete Physical Design Flow: RTL to GDSII.....	5
1.1 Why Physical Design Exists	5
1.2 The Two Major Phases.....	5
1.3 Flow Stage Map	6
1.4 Stage-by-Stage Narrative	6
1.4.1 RTL Design Entry	6
1.4.2 Logic Synthesis (Design Compiler)	6
1.4.3 Data Preparation (ICC2 Library Creation)	7
1.4.4 Multi-Corner/Multi-Mode (MCMM) Setup	7
1.4.5 Floorplanning.....	7
1.4.6 Pad Frame Construction	7
1.4.7 Power Plan.....	8
1.4.8 Placement.....	8
1.4.9 Clock Tree Synthesis (CTS).....	8
1.4.10 Routing	8
1.5 How the Scripts Relate to Each Other.....	8
Chapter 2 — common_setup.tcl: The Shared Configuration Backbone.....	10
2.1 Full Script Listing.....	10
2.2 Output Placeholder	17
2.3 Command-by-Command Explanation	17
2.3.1 Header and Design Identification	17
2.3.2 Hierarchical Flow Variables	18
2.3.3 Library and Technology Setup	18
2.3.4 Power/Ground Naming.....	20
2.3.5 Routing Layer Range.....	20
2.3.6 Floorplan Settings.....	21
2.3.7 Power Plan Settings	21
2.3.8 Placement Settings.....	21
2.3.9 Clock Tree Synthesis Settings	22
2.3.10 Closing.....	22
Chapter 3 — RTL Source: fifo_8bit.v and FIFO_ChipTop.v	23

Banothu Bapuji
VLSI PD RTL-TO GDS USING SYNOPSYS TOOLS

3.1 fifo_8bit.v — Full Listing	23
3.2 fifo_8bit.v — Explanation.....	25
3.3 Output Placeholder — RTL Simulation / Lint	26
3.4 FIFO_ChipTop.v — Full Listing	28
3.5 FIFO_ChipTop.v — Explanation.....	31
3.6 Output Placeholder — Elaboration / Schematic	32
Chapter 4 — dc_script.tcl: Logic Synthesis in Design Compiler	33
4.1 Full Script Listing.....	33
4.2 Output Placeholder	35
4.3 Command-by-Command Explanation	36
Chapter 5 — data_preparation.tcl: ICC2 Library Creation and Netlist Import	40
5.1 Full Script Listing.....	40
5.2 Output Placeholder	41
5.3 Command-by-Command Explanation	42
Chapter 6 — mmmm.tcl: Multi-Corner/Multi-Mode Setup.....	46
6.1 Full Script Listing.....	46
6.2 Output Placeholder	48
6.3 Command-by-Command Explanation	48
Chapter 7 — floorplan.tcl: Die/Core Floorplan Initialization	52
7.1 Full Script Listing.....	52
7.2 Output Placeholder	53
7.3 Command-by-Command Explanation	54
Chapter 8 — pad_frame.tcl: IO Pad Ring Construction	57
8.1 Full Script Listing.....	57
8.2 Output Placeholder	62
8.3 Command-by-Command Explanation	62
8.3.1 Setup and Restore	63
8.3.2 Section 1 — Port-to-Side Assignment.....	63
8.3.3 Section 2 — Signal Pad Discovery.....	64
8.3.4 Section 3 — Power Pad Discovery.....	64
8.3.5 Section 4 — Assignment to IO Guides	65
8.3.6 Section 5 — Corner Cells.....	65
8.3.7 Section 6 — Commit Placement	66
8.3.8 Section 7 — IO Power/Ground Net Connections.....	66

Banothu Bapuji
VLSI PD RTL-TO GDS USING SYNOPSYS TOOLS

8.3.9 Section 8 — IO Filler Insertion	66
8.3.10 Section 9 — Save	67
Chapter 9 — powerplan.tcl: Power Delivery Network Construction	68
9.1 Full Script Listing.....	68
9.2 Output Placeholder	70
9.3 Command-by-Command Explanation	70
Chapter 10 — placement.tcl: Standard Cell Placement.....	76
10.1 Full Script Listing.....	76
10.2 Output Placeholder	78
10.3 Command-by-Command Explanation.....	78
Chapter 11 — cts.tcl: Clock Tree Synthesis	82
11.1 Full Script Listing.....	82
11.2 Output Placeholder	83
11.3 Command-by-Command Explanation.....	84
Chapter 12 — route.tcl: Detailed Routing	88
12.1 Full Script Listing.....	88
12.2 Output Placeholder	89
12.3 Command-by-Command Explanation.....	90
12.4 End of Flow — Next Steps	94

Chapter 1 — The Complete Physical Design Flow: RTL to GDSII

Physical Design (PD), also called Place-and-Route (P&R) or Back-End Design, is the process that converts a gate-level or RTL description of a digital circuit into a manufacturable layout (GDSII) that respects timing, power, area, and foundry design rules. This book documents a complete, real, working RTL-to-GDSII flow for an 8-bit FIFO chip (FIFO_ChipTop), built on Synopsys Design Compiler (DC) for synthesis and Synopsys IC Compiler II (ICC2) for the physical implementation stages, targeting the SAED14nm_EDK_03_2025 educational/reference process design kit (HVT standard-cell library plus a wire-bond IO library).

Every chapter that follows is dedicated to one script from the flow. Each chapter reproduces the script exactly as supplied, followed by a command-by-command explanation, and a placeholder for the tool-output screenshot that should be captured when the step is actually run. Before diving into the scripts, this chapter lays out the conceptual flow end-to-end so that each script's place in the bigger picture is clear.

1.1 Why Physical Design Exists

RTL (Register Transfer Level) Verilog/VHDL describes circuit behavior — registers, combinational logic, and the operations performed on data each clock cycle. RTL by itself has no notion of silicon area, wire delay, cell drive strength, or routing congestion. The PD flow is the bridge that takes this purely logical description and produces an actual chip layout: a set of polygons on multiple silicon and metal layers that a foundry can fabricate. Along the way, the flow must guarantee that the chip will meet its timing requirements (setup/hold), stay within its power budget, fit in the targeted die area, and be physically routable and DRC/LVS clean.

1.2 The Two Major Phases

The flow used in this book is split into two major phases that use two different EDA tools, mirroring standard industry practice:

- Logic Synthesis (Design Compiler / DC) — translates technology-independent RTL into a technology-mapped gate-level netlist built from real standard cells in the SAED14nm HVT library, optimized for the timing/power/area constraints in the design's SDC.
- Physical Implementation (IC Compiler II / ICC2) — takes that gate-level netlist and physically realizes it: floorplanning, IO pad placement, power network construction, cell placement, clock tree synthesis, and detailed routing, finishing with a layout ready for signoff and GDSII streamout.

1.3 Flow Stage Map

The table below summarizes every stage covered in this book, the tool/script responsible, and the data that flows in and out of each stage.

Stage	Script	Key Inputs	Key Outputs
1. RTL Design	fifo_8bit.v / FIFO_ChipTop.v	Design specification	Verilog RTL + pad-level wrapper
2. Logic Synthesis	dc_script.tcl (+ common_setup.tcl)	RTL, Liberty .db, SDC constraints	Gate-level netlist, synthesis SDC
3. Library/Data Setup	data_preparation.tcl	Netlist, NDM libs, TLU+, MCMM	ICC2 design library (post_import_design)
4. MCMM Setup	mcmm.tcl	SDC, TLU+ corners	Scenarios: func_fast / func_slow
5. Floorplanning	floorplan.tcl	Die/core size, IO ring spec	Die + core area, PG pin connections
6. Pad Frame	pad_frame.tcl	IO ring guides, pad netlist instances	Placed and connected IO pad ring
7. Power Plan	powerplan.tcl	PG net/strap/rail settings	Power mesh, rails, vias (PG-DRC clean)
8. Placement	placement.tcl	Floorplan + PG, timing constraints	Legalized, timing-optimized placement
9. Clock Tree Synthesis	cts.tcl	Placed design, clock targets	Balanced clock tree, post-CTS timing
10. Routing	route.tcl	CTS'd design, routing rules	Fully routed, DRC-clean layout
11. Signoff / GDSII	(downstream — not in this script set)	Routed design	GDSII, DRC/LVS/STA signoff reports

1.4 Stage-by-Stage Narrative

1.4.1 RTL Design Entry

The design starts as two Verilog files. `fifo_8bit.v` is the synthesizable FIFO core: a 16-deep, 8-bit-wide synchronous FIFO with independent write-enable/read-enable control, a write pointer, read pointer, an occupancy counter, and full/empty flags. `FIFO_ChipTop.v` is the chip-level wrapper: it instantiates the FIFO core and surrounds every primary input/output with SAED14nm IO pad cells (ISH0818_TB for the clock, IO818_TB for inputs, DI0818_TB for outputs), because a real chip's pins are never connected directly to silicon — they must pass through electrostatic-discharge (ESD) protected, level-shifting IO pad cells that interface the chip's 1.62-1.96V IO ring to external package pins and the chip's internal 0.72-0.88V core logic to the rest of the die.

1.4.2 Logic Synthesis (Design Compiler)

dc_script.tcl reads the RTL, elaborates the design, applies the timing/power constraints from the chip's SDC, and runs compile_ultra to map the technology-independent logic onto real SAED14nm_EDK standard cells from the HVT (High-Threshold-Voltage, leakage-optimized) library. The result is a gate-level netlist (structural Verilog using real cell names) plus a post-synthesis SDC that captures the constraints needed downstream. common_setup.tcl is sourced first and supplies every path, library, and naming variable used throughout — both by DC and by every ICC2 script later in the flow — so that there is exactly one place to update technology/library information.

1.4.3 Data Preparation (ICC2 Library Creation)

data_preparation.tcl is the first ICC2 (IC Compiler II) script. It creates the ICC2 design library, reads in the synthesized gate-level netlist, links it against the NDM (Synopsys' compiled logical+physical library format) reference libraries, defines metal-layer routing directions, reads in the TLU+ parasitic technology files (for RC extraction during placement/routing-aware timing), and sources the multi-corner/multi-mode (MCMM) setup. The resulting design state is checkpointed as the post_import_design library and the import_done block — every later script restores from one of these saved checkpoints rather than re-running data preparation.

1.4.4 Multi-Corner/Multi-Mode (MCMM) Setup

mcmm.tcl defines the timing corners (slow/fast process-voltage-temperature corners) and scenarios (func_slow for setup-dominant analysis at 125°C, func_fast for hold-dominant analysis at -40°C) that every later optimization and signoff step in ICC2 will analyze against simultaneously. This is what makes the flow "multi-corner": the tool optimizes placement, CTS, and routing so that timing holds across both the slow/hot and fast/cold extremes a real chip will experience in the field, not just a single nominal condition.

1.4.5 Floorplanning

floorplan.tcl establishes the physical canvas: the chip's die boundary, the core area offset that reserves space for the IO ring, and creates the four IO-ring "guides" (top/bottom/left/right) that pad_frame.tcl will later populate with pad cells. It also makes the first power/ground connections at the std-cell pin level (VDD/VSS plus the FinFET body-bias pins VBP/VBN).

1.4.6 Pad Frame Construction

pad_frame.tcl populates the IO ring guides created in floorplanning with the actual pad cell instances. Because this design's RTL already instantiates every IO pad (signal pads and power pads alike) directly in FIFO_ChipTop.v, this script does not create new cells — it discovers the existing pad instances (by tracing each top-level port's net back to its PADIO pin), assigns them to a ring side, creates the four IO corner cells, commits the physical placement of the ring with

place_io, makes the IO-domain power/ground connections, and fills any remaining ring gaps with IO filler cells.

1.4.7 Power Plan

powerplan.tcl builds the on-chip power delivery network: it creates the VDD/VSS nets and top-level inout ports, connects every cell's power pins logically, builds a horizontal/vertical power-ground mesh on mid-stack metal layers (compile_pg), creates the M1 standard-cell power rails that every cell sits on, and finally verifies the result is both electrically connected (check_pg_connectivity) and DRC-clean (check_pg_drc) before checkpointing as powerplan_done.

1.4.8 Placement

placement.tcl performs timing-driven coarse placement of every standard cell (create_placement -timing_driven), applies the parasitic estimation corners set up earlier, legalizes the placement so every cell sits on a legal site row with no overlaps (legalize_placement), and then runs place_opt to perform placement-aware timing/power/area optimization. The result, checkpointed as placement_done, is a fully legal, optimized cell placement ready for clock tree synthesis.

1.4.9 Clock Tree Synthesis (CTS)

cts.tcl builds the physical clock distribution network. It defines non-default routing rules (NDR) for clock nets (wider/more-spaced wires than signal nets, to reduce clock-network resistance and crosstalk sensitivity), restricts which library cells CTS may use as clock buffers, sets target latency and skew goals, and runs clock_opt to synthesize a balanced clock tree that meets those goals across all MCM scenarios. PG pin connections are re-verified and the result is saved as cts_done.

1.4.10 Routing

route.tcl performs the final detailed routing of every signal net on the die: global routing, track assignment, and detailed routing, all timing- and crosstalk-driven, restricted to the metal layer range defined in common_setup.tcl. route_auto produces the initial route (checkpointed as initial_route), and route_opt then performs post-route optimization to fix any remaining timing or DRC violations, with the final state saved as route_done. From here, the design proceeds to signoff (STA, physical verification, IR-drop analysis) and GDSII streamout — steps outside the script set covered in this book, but the natural next step after route_done.

1.5 How the Scripts Relate to Each Other

Every ICC2-stage script in this flow follows the same pattern: it begins by sourcing common_setup.tcl to pull in every shared variable (paths, library names, layer assignments,

floorplan/power/placement/CTS numeric settings), then restores the previous stage's saved checkpoint block, performs its stage-specific work, and finishes by saving a new named checkpoint block plus the ICC2 design library. This checkpoint-based structure means any stage can be re-run independently by restoring the prior stage's saved block, which is exactly why each script in this book is presented and explained as a standalone, complete unit.

The remaining chapters proceed in flow order: `common_setup.tcl` (the shared configuration backbone) is presented first since every other script depends on it, followed by the RTL source files, then `dc_script.tcl`, and then every ICC2 stage script in execution order through to `route.tcl`.

Chapter 2 — common_setup.tcl: The Shared Configuration Backbone

common_setup.tcl is the single source of truth for every variable used across the entire flow — by Design Compiler during synthesis and by every IC Compiler II script during physical implementation. It is sourced at the top of dc_script.tcl, data_preparation.tcl, floorplan.tcl, pad_frame.tcl, powerplan.tcl, placement.tcl, cts.tcl, and route.tcl. Centralizing these settings in one file means a technology, library, or design-name change only has to be made once, and it is the reason every downstream script in this book is so short: nearly all of their configuration lives here.

This particular file also doubles as a running engineering log: many comments record decisions that were verified against the official SAED14nm_EDK databooks (Digital Standard Cell Libraries Databook, General I/O Libraries Databook, and the EDK/Technology Data Databook), including one correction to an earlier wrong assumption about metal-layer routing directions. Reading those comments alongside the code is part of understanding why each value was chosen, not just what it is.

2.1 Full Script Listing

```
puts "RM-Info: Running script [info script]\n"

#####
#####
# Variables common to all flow scripts (DC synthesis + ICC2
# floorplan/powerplan/place/cts/route)
# Script: common_setup.tcl
#
# Targets SAED14nm_EDK_03_2025, HVT std-cell library + WireBond IO library.
#
# STATUS: Confirmed against the official SAED14nm_EDK databooks (Digital
# Standard Cell
# Libraries Databook, General I/O Libraries Databook, EDK and Technology Data
# Databook) -
# this replaces earlier guesses, including ONE THAT WAS WRONG: routing directions
# are
# M1=V/M2=H/M3=V/... not the M1=H/M2=V/... originally assumed.
#####
#####

#####
#####
# 1. Design identification
#####
#####
```

```

set DESIGN_NAME          "FIFO_ChipTop"
set CORE_DESIGN_NAME     "fifo_8bit"
set ICC2_LIB_NAME        "./lib"

# NOTE: a "${CORE_DESIGN_NAME}_defines.v" entry was here in earlier versions
# of this
# file but was never actually created - it was a placeholder filename inherited from the
# original flow template, not real content either of us wrote. Removed: chiplet_puf_top.v
# compiled cleanly with zero undefined-macro errors/warnings, confirming it isn't
# needed.
# Re-add a real file here if you later author actual `define content for this design.
set RTL_SOURCE_FILES     "\
    ./rtl/${CORE_DESIGN_NAME}.v \
    ./rtl/${DESIGN_NAME}.v \
"

set NETLIST_FILE          "./${DESIGN_NAME}_netlist.v"
set GTECH_NETLIST_FILE   "./${DESIGN_NAME}_gtech.vs"
set SDC_OUTPUT_FILE       "./${DESIGN_NAME}.sdc"
set SVF_FILE              "${DESIGN_NAME}.svf"
set CONSTRAINTS_FILE      "./FIFO_ChipTop.sdc"
set MCMM_FILE             "./mcmml.tcl"
set SEARCH_PATH           ". ./rtl"

#####
#####
# 2. Hierarchical Flow Design Variables (leave blank for flat designs)
#####
#####

set HIERARCHICAL_DESIGNS  ""
set HIERARCHICAL_CELLS    ""

#####
#####
# 3. Library / Technology Setup (SAED14nm_EDK_03_2025)
#####
#####

set EDK_ROOT              "/home/vlsi/c2s_tools/SAED14nm_EDK_03_2025"
set STD_HVT_PATH          "${EDK_ROOT}/SAED14nm_EDK_STD_HVT"
set IO_REF_PATH           "${EDK_ROOT}/SAED14nm_EDK_IO_WB"

```

```

set DESIGN_REF_TECH_PATH
"${EDK_ROOT}/SAED14nm_EDK_TECH_DATA"

# CONFIRMED (Digital Standard Cell Libraries Databook, Section 1.7 naming
convention):
# cell prefix is SAED<VT>14, so HVT -> SAEDHVT14. Used to build
boundary/tap/filler
# references below.
set CELL_PREFIX          "SAEDHVT14"

# Liberty corner used by DC for SYNTHESIS MAPPING. Only std cells - IO pads are
# structural fixed instances in the chip-top RTL wrapper (not something DC should ever
# map general logic onto), so they go in IO_LIBRARY_FILES/link_library below
instead,
# not here. ss0p72v125c CONFIRMED as a real characterized corner name (std-cell
databook
# Table 1.8); IO databook Table 1.6 confirms the matching IO corner at 1.62V IO
supply.
set TARGET_LIBRARY_FILES  " \
${STD_HVT_PATH}/liberty/nldm/base/saed14hvt_base_ss0p72v125c.db "

# IO pad Liberty - needed only so `link` can resolve the I0818/DI0818/ISH0818
instances
# in Chiplet_PUF_ChipTop.v. Not part of target_library.
set IO_LIBRARY_FILES      " \
${IO_REF_PATH}/liberty/nldm/saed14_io_wb_ss0p72v125c_1p62v.db "

set LINK_LIBRARY_FILES    "* ${TARGET_LIBRARY_FILES}
${IO_LIBRARY_FILES}"

# Confirmed present: saed14hvt_base_frame_timing.ndm /
saed14io_wb_frame_timing.ndm are
# real compiled NDM libraries (logical + physical bound together).
set NDM_REFERENCE_FILES  ""
lappend NDM_REFERENCE_FILES
"${STD_HVT_PATH}/ndm/saed14hvt_base_frame_timing.ndm"
lappend NDM_REFERENCE_FILES
"${IO_REF_PATH}/ndm/saed14io_wb_frame_timing.ndm"

set MW_REFERENCE_CONTROL_FILE  ""

set TECH_FILE             "${DESIGN_REF_TECH_PATH}/tf/saed14nm_1p9m.tf"
set MAP_FILE
"${DESIGN_REF_TECH_PATH}/map/saed14nm_tf_itf_tduplus.map"

```

```

set TLUPLUS_MAX_FILE
"${DESIGN_REF_TECH_PATH}/tlup/saed14nm_1p9m_Cmax.tlup"
set TLUPLUS_MIN_FILE
"${DESIGN_REF_TECH_PATH}/tlup/saed14nm_1p9m_Cmin.tlup"
# Cnom (nominal) also available at
${DESIGN_REF_TECH_PATH}/tlup/saed14nm_1p9m_Cnom.tlup
# if a third (typical) parasitic corner is wanted later.

set PARASITIC_EARLY_SPEC_NAME "best_para"
set PARASITIC_LATE_SPEC_NAME "worst_para"

# Only one Liberty corner (ss0p72v125c) is loaded, so MIN and MAX point at the same
# operating condition - there's only one PVT point available right now.
# CONFIRMED as a real characterized corner name (std-cell databook Table 1.8:
"ss0p72v125c,
# Slow-Slow, 0.72V, 125C"). The databook doesn't show the literal Liberty
# `operating_conditions` group name verbatim, so this is high-confidence but not
# byte-for-byte verified - if set_operating_conditions errors, check with:
# grep -A5 "operating_conditions"
${STD_HVT_PATH}/liberty/nldm/base/saed14hvt_base_ss0p72v125c.lib
set OPERATING_CONDITION_MIN "ss0p72v125c"
set OPERATING_CONDITION_MAX "ss0p72v125c"

# Wire load models are a pre-placement estimation technique mostly relevant to older/
# larger nodes; modern flows (and SAED14nm's NLDM kit) skip them in favor of
TLU+/
# placement-aware parasitic estimation. Left unset - the WLM lines in dc_script.tcl are
# commented out accordingly.
set WIRE_LOAD_MODEL ""
set WIRE_LOAD_MODE "top"

#####
#####
# 4. Power / Ground Naming
#####
#####

# CONFIRMED (Digital Standard Cell Libraries Databook 6.5, General I/O Libraries
Databook
# 7.3): std cells use VDD/VSS (+ VBP/VBN FinFET body-bias pins, tied to VDD/VSS -
handled
# in floorplan.tcl). IO pads additionally need VDDIO/VSSIO (separate I/O supply rail,
# 1.62-1.96V vs core's 0.72-0.88V) - handled in pad_frame.tcl.
set NDM_POWER_NET "VDD"

```

```

set NDM_POWER_PORT      "VDD"
set NDM_GROUND_NET      "VSS"
set NDM_GROUND_PORT     "VSS"
set IO_POWER_NET        "VDDIO"
set IO_GROUND_NET       "VSSIO"

#####
#####
# 5. Routing Layer Range
# CONFIRMED (Digital Standard Cell Libraries Databook, Table 6.1 "Routing
# directions for
# signal metal layers"). This OVERTURNS an earlier guess: the real alternation is
# M1=vertical, M2=horizontal, M3=vertical, M4=horizontal, M5=vertical,
# M6=horizontal,
# M7=vertical, M8=horizontal, M9=vertical - opposite of what was assumed before this
# databook was available.
#####
#####

set MIN_ROUTING_LAYER    "M2"    ;# M1 reserved/ignored for general
routing -
                                ;# CONFIRMED: databook 6.3 explicitly recommends
                                ;# `set_ignored_layers -min_routing_layer M2`
set MAX_ROUTING_LAYER    "M9"
set HORIZONTAL_LAYERS     "M2 M4 M6 M8"
set VERTICAL_LAYERS       "M1 M3 M5 M7 M9"

# --- PNR-mandated settings for this library (Digital Standard Cell Libraries Databook,
# Section 6: Physical Design Notes / Routing options) ---
# Not yet consumed by route.tcl (doesn't exist yet) - captured here so they aren't lost.
set FLOORPLAN_SEGMENT_RULE "horizontal_even" ;# REQUIRED before
initialize_floorplan
set ROUTE_WIRE_ON_GRID_BY_LAYER {{M1 true} {M2 true} {M3 true}}
set ROUTE_VIA_ON_GRID_BY_LAYER  {{VIA1 false} {VIA2 true}}
set ROUTE_IGNORED_MIN_LAYER     "M2" ;# set_ignored_layers -
min_routing_layer M2

# Spacing-rule files - only needed if this design ends up using always-on (pg/cg/iso/ret)
# or level-shifter (dlvl/ulvl) cells. Not currently used by this design (no UPF/
# multi-voltage logic), but paths captured here in case that changes.
set AO_SPACING_RULES_FILE
"${STD_HVT_PATH}/spacing_rules/AO_spacing_rules_icc2_fc.tcl"
set LVL_SPACING_RULES_FILE
"${STD_HVT_PATH}/spacing_rules/LVL_spacing_rules_icc2_fc.tcl"

```

```
#####
#####
# 6. Floorplan Settings
#####
#####

# TODO: still SCL180-era placeholder magnitudes - re-derive real die size from actual
# gate count + SAED14 IO cell pitch (108um pad height/width per the IO databook)
before
# tapeout.
set DIE_WIDTH          2000
set DIE_HEIGHT         2000
set CORE_OFFSET        500
set IO_RING_NAME        "io_ring"

set CORE_UTILIZATION    0.5
set CORE_ASPECT_RATIO   {1 1}

# PIN_LAYERS / OUTPUT_PIN_SIDE / INPUT_PIN_SIDE / INOUT_PIN_SIDE
removed: those applied to
# exporting this block as an abstract (LEF) macro for a higher hierarchy level. This
design
# IS the full chip with a wire-bond pad ring, not a sub-block - ports are realized as
# physical pads (handled in pad_frame.tcl), not abstracted LEF pins. Not applicable here.

# CONFIRMED (Digital Standard Cell Libraries Databook, Section 6.1 Floorplanning):
# - top/bottom corner cells: *_CAP*IN13
# - top boundary cells: *_CAPT2 and *_CAPT3
# - bottom boundary cells: *_CAPB2 and *_CAPB3
# - left/right boundary cells: *_CAPT2 and *_CAPB2 (same cells, different orientation)
# - recommended max TAP cell spacing: 30um
set CORNER_CELL_TOP      "${CELL_PREFIX}_CAPTIN13"
set CORNER_CELL_BOTTOM   "${CELL_PREFIX}_CAPBIN13"
set TOP_BOUNDARY_CELLS   "${CELL_PREFIX}_CAPT2
${CELL_PREFIX}_CAPT3"
set BOTTOM_BOUNDARY_CELLS "${CELL_PREFIX}_CAPB2
${CELL_PREFIX}_CAPB3"
set LEFT_RIGHT_BOUNDARY_CELLS "${CELL_PREFIX}_CAPT2
${CELL_PREFIX}_CAPB2"
# Three TAP cell variants exist (TAPDS/TAPPN/TAPPP10); the databook gives a max-
spacing
# rule but doesn't say which is the generic default. TAPDS used as a reasoned default -
# verify against your course's expected usage if this matters.
```

```

set TAP_CELL_LIB_CELL      "${CELL_PREFIX}_TAPDS"
set TAP_CELL_DISTANCE      30

# Std-cell filler insertion: per the databook, DISCOVER filler cells dynamically rather
# than hardcoding names (more robust across library updates) - this is embedded directly
# in floorplan.tcl rather than as a static list here:
# set FILLER_CELLS [get_object_name [sort_collection -descending \
#   [get_lib_cells */*_FILL* -filter "name !~ *_LVLFILL* AND name !~ *FILLP*
#   AND \
#     name !~ *ECO* AND name !~ *Y2* AND name !~ *SPACER*"] area]]
# create_stdcell_fillers -lib_cells ${FILLER_CELLS}
# IO pad-ring filler insertion uses a similar but separate official pattern (different
# cell family, different exclusion rule for TB vs LR) - embedded directly in
# pad_frame.tcl, see the IO Libraries Databook Section 7.1 usage example.

#####
#####
# 7. Power Plan Settings
#####
#####

set PG_STRAP_LAYER_H      "M6"  ;# horizontal per Table 6.1; M6 chosen as a
                                ;# mid-stack strap layer - TODO VERIFY against
                                ;# real congestion/IR-drop needs
set PG_STRAP_LAYER_V      "M5"  ;# vertical per Table 6.1
set PG_STRAP_WIDTH        "0.3" ;# TODO VERIFY against SAED14 DRC
                                minimums
set PG_STRAP_PITCH        "4"
set PG_STRAP_OFFSET       "1.6"

# CONFIRMED (Digital Standard Cell Libraries Databook, Section 2, Table 2.2 and Fig
# 2.1):
# std-cell power/ground rails sit on M1, rail width 0.094um.
set STD_CELL_RAIL_LAYER    "M1"
set STD_CELL_RAIL_WIDTH    "0.094"

#####
#####
# 8. Placement Settings
#####
#####

set PLACEMENT_MAX_DENSITY  0.6
set PLACEMENT_CONGESTION_MAX_UTIL  0.6

```

```
#####
#####
# 9. Clock Tree Synthesis Settings
#####
#####

# No cell in the SAED14 HVT databook is explicitly labeled "use this for CTS".
Reasoned
# pick (not an official PDK requirement, unlike the boundary/tap/filler cells above):
# CLKSPLT (dedicated "Clock Splitter" cell) plus the symmetric-rise/fall buffer family
# (BUF_S_*), since duty-cycle symmetry matters for clock distribution. Verify/adjust
once
# an actual cts.tcl is written.
set CTS_BUFFER_CELLS      "*${CELL_PREFIX}_CLKSPLT_*"
                           "${CELL_PREFIX}_BUF_S_*"
set CTS_MAX_FANOUT        30
set CTS_TARGET_LATENCY    0.050
set CTS_TARGET_SKEW       0.030

set CTS_NDR_LAYERS        "M1 M2" ;# lower layers for local clock distribution
set CTS_NDR_WIDTH         "0.3"   ;# TODO VERIFY against this kit's min
width/spacing
set CTS_NDR_SPACING       "0.3"   ;# TODO VERIFY

# Top two of the 9 confirmed metal layers - lowest resistance for best skew/insertion
# delay (same reasoning as before, layer numbers unaffected by the H/V direction fix).
set CTS_MIN_ROUTING_LAYER "M8"
set CTS_MAX_ROUTING_LAYER "M9"

#####
#####
#set LIBRARY_DONT_USE_FILE  ".ref/use_tie.tcl"

puts "RM-Info: Completed script [info script]\n"
```

2.2 Output Placeholder

common_setup.tcl is sourced (not run standalone) — capture the console output of the first script that sources it (data_preparation.tcl or dc_script.tcl) to confirm all paths resolved without error.

2.3 Command-by-Command Explanation

2.3.1 Header and Design Identification

puts "RM-Info: Running script [info script]\n" Prints a standardized banner showing the script's own filename (via the built-in [info script] command), used as a marker in log files so every stage's start is easy to find when reviewing a run log.

set DESIGN_NAME "FIFO_ChipTop" Defines the top-level (chip-level) module name. This is the name DC and ICC2 will use as current_design throughout the flow — it must exactly match the top module name in the Verilog RTL.

set CORE_DESIGN_NAME "fifo_8bit" Names the FIFO core sub-module instantiated inside the chip-level wrapper. Kept separate from DESIGN_NAME because the core RTL file and the chip-top RTL file are two different source files that both need to be read during synthesis.

set ICC2_LIB_NAME ".lib" Sets the filesystem path where the ICC2 design library (an NDM-format library holding the design's logical and physical data) will be created by data_preparation.tcl's create_lib command.

set RTL_SOURCE_FILES " ./rtl/\${CORE_DESIGN_NAME}.v ./rtl/\${DESIGN_NAME}.v " Builds the list of Verilog files to be read during synthesis, using Tcl variable substitution so the actual filenames depend on the two design-name variables above. Order matters for hierarchical compiles — the core module is listed before the chip-top wrapper that instantiates it.

set NETLIST_FILE / GTECH_NETLIST_FILE / SDC_OUTPUT_FILE / SVF_FILE / CONSTRAINTS_FILE / MCMM_FILE / SEARCH_PATH Define every file path used by the synthesis stage: the post-synthesis gate-level netlist output, the intermediate technology-independent GTECH netlist (useful for debugging structural issues before technology mapping), the post-synthesis SDC output, the SVF (Synopsys Verification File, used for formal equivalence checking against the original RTL), the input SDC constraints file, the MCMM corner/scenario setup script, and the Tcl search path DC uses to locate RTL files referenced by read_verilog.

2.3.2 Hierarchical Flow Variables

set HIERARCHICAL_DESIGNS "" / set HIERARCHICAL_CELLS "" Left blank because this is a flat design (synthesized and placed as one unit, not partitioned into separately-compiled hierarchical blocks). These variables exist in the template to support hierarchical flows where individual sub-blocks are synthesized independently and later assembled — not needed for a design this small.

2.3.3 Library and Technology Setup

set EDK_ROOT / STD_HVT_PATH / IO_REF_PATH / DESIGN_REF_TECH_PATH Define the root installation path of the SAED14nm_EDK_03_2025 process design kit and three sub-paths within it: the HVT standard-cell library, the wire-bond IO library, and the technology/TLU+ data directory. Every later library and tech-file path is built from these four roots.

set CELL_PREFIX "SAEDHVT14" Captures the standard-cell library's naming prefix (confirmed against the databook's Section 1.7 naming convention) so that every later reference to a specific cell family — boundary cells, tap cells, filler cells, dont-use exclusions — can be built with simple string substitution instead of being hardcoded individually.

set TARGET_LIBRARY_FILES
"\${STD_HVT_PATH}/liberty/nldm/base/saed14hvt_base_ss0p72v125c.db" Specifies the Liberty timing library DC will map logic onto during synthesis — the slow-slow, 0.72V, 125 degC (ss0p72v125c) corner. Only the standard-cell library is listed here, deliberately excluding IO pads, because DC should never select a pad cell as a general logic-mapping target.

set IO_LIBRARY_FILES
"\${IO_REF_PATH}/liberty/nldm/saed14_io_wb_ss0p72v125c_1p62v.db" Specifies the IO pad library's matching Liberty corner at 1.62V IO supply. This is needed only so the link command can resolve the IO pad instances (IO818_TB, DI0818_TB, ISH0818_TB) already present in the chip-top RTL — it is kept out of target_library for the same reason as above.

set LINK_LIBRARY_FILES "*" \${TARGET_LIBRARY_FILES} \${IO_LIBRARY_FILES}" Builds the full link_library list DC and ICC2 use to resolve every cell reference in the design — the leading asterisk tells the tool to also search the design's own already-compiled cells, followed by both the std-cell and IO Liberty files.

set NDM_REFERENCE_FILES ... lappend ... Builds a Tcl list of the two pre-compiled NDM (Synopsys' combined logical+physical Milkyway-successor format) reference libraries — one for the HVT std cells, one for the IO pads — that ICC2's create_lib will use as -ref_libs so physical (LEF-equivalent) data is available alongside timing data.

set MW_REFERENCE_CONTROL_FILE "" Left blank — a Milkyway reference control file is not needed because this flow uses the newer NDM library format exclusively, not legacy Milkyway libraries.

set TECH_FILE / MAP_FILE / TLUPLUS_MAX_FILE / TLUPLUS_MIN_FILE Define the ICC2 technology file (process layer stack definition), the TLU+ layer-mapping file, and the two TLU+ parasitic technology files used for RC extraction at the max (Cmax, worst-case capacitance) and min (Cmin, best-case capacitance) corners. A third Cnom file exists in the kit but is not wired in, since only two parasitic corners are used in this flow.

set PARASITIC_EARLY_SPEC_NAME "best_para" / set PARASITIC_LATE_SPEC_NAME "worst_para" Names the two parasitic specifications referenced later wherever ICC2 needs to know which RC extraction corner to use for early-mode (best-case, hold-relevant) versus late-mode (worst-case, setup-relevant) timing analysis.

set OPERATING_CONDITION_MIN "ss0p72v125c" / set OPERATING_CONDITION_MAX "ss0p72v125c" Sets the Liberty operating condition for

both min and max delay analysis to the same corner, because only one Liberty corner (ss0p72v125c) is currently loaded into target_library — there is no second PVT point available yet to give MIN and MAX genuinely different values.

set WIRE_LOAD_MODEL "" / set WIRE_LOAD_MODE "top" Wire load models are left unset because SAED14nm's NLDM Liberty kit ships no wire_load_table data; modern flows at this node rely on TLU+ and placement-aware parasitic estimation instead. WIRE_LOAD_MODE is set for completeness but is unused while WIRE_LOAD_MODEL is blank.

2.3.4 Power/Ground Naming

set NDM_POWER_NET / NDM_POWER_PORT "VDD" ; set NDM_GROUND_NET / NDM_GROUND_PORT "VSS" Standardizes the core power and ground net/port names used by every PG-related command in floorplan.tcl, powerplan.tcl, placement.tcl, and cts.tcl, confirmed against the std-cell databook's power/ground pin naming (Section 6.5).

set IO_POWER_NET "VDDIO" / set IO_GROUND_NET "VSSIO" Names the separate IO-domain power/ground rails (1.62-1.96V) required by the wire-bond pad library, distinct from the 0.72-0.88V core supply — used exclusively in pad_frame.tcl.

2.3.5 Routing Layer Range

set MIN_ROUTING_LAYER "M2" / set MAX_ROUTING_LAYER "M9" Defines the usable general-routing metal stack: M1 is excluded from general routing (reserved for standard-cell internal rails/pins) per the databook's explicit recommendation to set_ignored_layers with -min_routing_layer M2, and M9 is the top of the available 9-layer stack.

set HORIZONTAL_LAYERS "M2 M4 M6 M8" / set VERTICAL_LAYERS "M1 M3 M5 M7 M9" Captures the confirmed per-layer preferred routing direction from the databook's Table 6.1 — alternating, with odd metal layers vertical and even metal layers horizontal. This list directly drives the routing_direction attribute assignment performed in data_preparation.tcl.

set FLOORPLAN_SEGMENT_RULE "horizontal_even" A PNR-mandated setting for this library that must be applied via set_app_options before initialize_floorplan is called in floorplan.tcl — it constrains how floorplan row segments are generated to match the library's site/row requirements.

set ROUTE_WIRE_ON_GRID_BY_LAYER / ROUTE_VIA_ON_GRID_BY_LAYER / ROUTE_IGNORED_MIN_LAYER Capture additional routing-grid and ignored-layer settings recommended by the databook's PNR notes, recorded here for the router stage to consume.

set AO_SPACING_RULES_FILE / LVL_SPACING_RULES_FILE Point to spacing-rule Tcl files needed only if the design later adopts always-on power-gating or level-shifter cells (UPF-driven multi-voltage logic). Not used by this design today since it has no UPF/multi-voltage requirements, but kept available for future reuse.

2.3.6 Floorplan Settings

set DIE_WIDTH 2000 / set DIE_HEIGHT 2000 / set CORE_OFFSET 500 Define the die's outer boundary (2000 x 2000 in the design's distance units) and the offset from the die edge to the core area, which reserves the ring space `pad_frame.tcl` will populate. Flagged in the comments as still placeholder magnitudes inherited from an earlier SCL180 version of the flow, pending re-derivation from actual gate count and the SAED14 IO cell pitch.

set IO_RING_NAME "io_ring" Names the IO ring object that `floorplan.tcl` creates with `create_io_ring` and that `pad_frame.tcl` references throughout (`io_ring.top`, `io_ring.bottom`, etc.).

set CORE_UTILIZATION 0.5 / set CORE_ASPECT_RATIO {1 1} Define target core utilization (50% — the fraction of core area expected to be occupied by standard cells) and a square (1:1) core aspect ratio, both reference values for floorplanning even though this script ultimately drives sizing from `DIE_WIDTH/DIE_HEIGHT` directly.

set CORNER_CELL_TOP / CORNER_CELL_BOTTOM / TOP_BOUNDARY_CELLS / BOTTOM_BOUNDARY_CELLS / LEFT_RIGHT_BOUNDARY_CELLS Define the exact boundary/corner/end-cap cell names required at the edges of every standard-cell row, confirmed against the databook's Section 6.1 floorplanning guidance (e.g. `*_CAPTIN13` for top/bottom corners, `*_CAPT2/*_CAPT3` for top edges).

set TAP_CELL_LIB_CELL "\${CELL_PREFIX}_TAPDS" / set TAP_CELL_DISTANCE 30 Selects the well/substrate TAP cell variant (TAPDS, a reasoned default among three available variants) and the maximum recommended spacing between tap cells (30 micrometers per the databook), used to prevent latch-up by ensuring every transistor is within range of a substrate contact.

2.3.7 Power Plan Settings

set PG_STRAP_LAYER_H "M6" / set PG_STRAP_LAYER_V "M5" Choose M6 (horizontal) and M5 (vertical) as the mid-stack metal layers for the power/ground mesh straps, consistent with the confirmed per-layer routing directions. Flagged for verification against real congestion/IR-drop analysis once available.

set PG_STRAP_WIDTH "0.3" / set PG_STRAP_PITCH "4" / set PG_STRAP_OFFSET "1.6" Set the strap width, repeat pitch, and starting offset (in micrometers) for the power mesh pattern built later in `powerplan.tcl`'s `create_pg_mesh_pattern`.

set STD_CELL_RAIL_LAYER "M1" / set STD_CELL_RAIL_WIDTH "0.094" Confirmed from the databook (Section 2, Table 2.2 and Figure 2.1): standard-cell power/ground rails sit on M1 with a rail width of 0.094 micrometers — used by `powerplan.tcl`'s `create_pg_std_cell_conn_pattern`.

2.3.8 Placement Settings

set PLACEMENT_MAX_DENSITY 0.6 / set PLACEMENT_CONGESTION_MAX_UTIL 0.6 Cap coarse placement at 60% maximum density in uncongested areas and 60% maximum utilization in congestion-driven spreading, giving the placer headroom to avoid overly dense regions that would later cause routing congestion.

2.3.9 Clock Tree Synthesis Settings

set CTS_BUFFER_CELLS "*/\${CELL_PREFIX}_CLKSPLT_*/\${CELL_PREFIX}_BUF_S_* Restricts which library cells clock_opt may use to build the clock tree to the dedicated clock-splitter cell family and the symmetric-rise/fall buffer family, chosen because duty-cycle symmetry matters for clock distribution even though no cell in the databook is explicitly labeled as the required CTS cell.

set CTS_MAX_FANOUT 30 / set CTS_TARGET_LATENCY 0.050 / set CTS_TARGET_SKEW 0.030 Set the maximum fanout any clock buffer may drive (30), and the target clock insertion latency (50 picoseconds) and target skew (30 picoseconds) goals that clock_opt will try to achieve.

set CTS_NDR_LAYERS "M1 M2" / set CTS_NDR_WIDTH "0.3" / set CTS_NDR_SPACING "0.3" Define the non-default routing rule used for local clock distribution on the lower metal layers, with wider width and spacing than minimum-pitch signal routing to reduce resistance and crosstalk sensitivity on clock nets.

set CTS_MIN_ROUTING_LAYER "M8" / set CTS_MAX_ROUTING_LAYER "M9" Restrict the global clock trunk routing to the top two metal layers, chosen for their lowest resistance per unit length, minimizing skew and insertion delay on the longest clock runs.

2.3.10 Closing

#set LIBRARY_DONT_USE_FILE "./ref/use_tie.tcl" A commented-out placeholder for an external dont_use exclusion file; not active in the current flow since the equivalent dont_use rules are applied directly in dc_script.tcl instead.

puts "RM-Info: Completed script [info script]\n" Prints the matching completion banner for the script-start banner at the top of the file, closing out the log entry for this source operation.

Chapter 3 — RTL Source: `fifo_8bit.v` and `FIFO_ChipTop.v`

The flow begins with two Verilog files: `fifo_8bit.v`, the synthesizable FIFO core, and `FIFO_ChipTop.v`, the chip-level pad wrapper that surrounds the core with SAED14nm IO pad cells. Both are read by `dc_script.tcl` during synthesis (`RTL_SOURCE_FILES` in `common_setup.tcl` lists the core file first, then the chip-top wrapper, since the wrapper instantiates the core).

3.1 `fifo_8bit.v` — Full Listing

```
module fifo_8bit #(
    parameter DATA_WIDTH = 8,
    parameter DEPTH = 16,
    parameter ADDR_WIDTH = 4
)(
    input wire      clk,
    input wire      rst,

    input wire      wr_en,
    input wire      rd_en,
    input wire [DATA_WIDTH-1:0] data_in,

    output reg [DATA_WIDTH-1:0] data_out,
    output wire      full,
    output wire      empty
);

    // FIFO Memory
    reg [DATA_WIDTH-1:0] mem [0:DEPTH-1];

    // Read and Write Pointers
    reg [ADDR_WIDTH-1:0] wr_ptr;
    reg [ADDR_WIDTH-1:0] rd_ptr;

    // Counter
    reg [ADDR_WIDTH:0] count;

    // Full and Empty Conditions
    assign full = (count == DEPTH);
    assign empty = (count == 0);

    integer i;
```

```

always @(posedge clk or posedge rst)
begin
    if (rst)
    begin
        wr_ptr  <= 0;
        rd_ptr  <= 0;
        count   <= 0;
        data_out <= 0;

        for(i = 0; i < DEPTH; i = i + 1)
            mem[i] <= 0;
    end
    else
    begin
        case ({wr_en, rd_en})

            // Write Only
            2'b10:
            begin
                if (!full)
                begin
                    mem[wr_ptr] <= data_in;
                    wr_ptr <= wr_ptr + 1'b1;
                    count <= count + 1'b1;
                end
            end

            // Read Only
            2'b01:
            begin
                if (!empty)
                begin
                    data_out <= mem[rd_ptr];
                    rd_ptr <= rd_ptr + 1'b1;
                    count <= count - 1'b1;
                end
            end

            // Simultaneous Read and Write
            2'b11:
            begin
                if (!full && !empty)
                begin

```

```

        mem[wr_ptr] <= data_in;
        data_out <= mem[rd_ptr];

        wr_ptr <= wr_ptr + 1'b1;
        rd_ptr <= rd_ptr + 1'b1;
    end
end

default:
begin
    // No Operation
end
endcase
end
end

endmodule

```

3.2 fifo_8bit.v — Explanation

module fifo_8bit #(parameter DATA_WIDTH = 8, parameter DEPTH = 16, parameter ADDR_WIDTH = 4) Declares the FIFO core module with three parameters: an 8-bit data width, a memory depth of 16 entries, and a 4-bit address width (enough to index 16 locations). Making these parameters rather than hardcoded constants allows the FIFO to be resized for other instantiations without editing the RTL body.

input wire clk, input wire rst The synchronous clock and an active-high asynchronous reset, both used in the single always block that drives all sequential state.

input wire wr_en, input wire rd_en, input wire [DATA_WIDTH-1:0] data_in Write-enable and read-enable control signals, and the 8-bit data input bus written into the FIFO on a write operation.

output reg [DATA_WIDTH-1:0] data_out, output wire full, output wire empty data_out is a registered output holding the most recently read word; full and empty are combinational derived status flags.

reg [DATA_WIDTH-1:0] mem [0:DEPTH-1] Declares the FIFO storage itself as a 16-entry array of 8-bit registers — this becomes 16 x 8 flip-flops (or, more realistically in a synthesis flow, gets technology-mapped to flip-flop-based memory rather than a dedicated SRAM macro, since no memory compiler is referenced in this flow).

reg [ADDR_WIDTH-1:0] wr_ptr, rd_ptr The write pointer and read pointer, each 4 bits wide, indexing the next memory location to write to or read from respectively. They wrap around naturally via unsigned overflow since ADDR_WIDTH (4 bits) exactly matches DEPTH (16).

reg [ADDR_WIDTH:0] count A 5-bit occupancy counter (one bit wider than the pointers) so it can represent the full range 0 through 16 inclusive, used to derive the full/empty flags without comparing wr_ptr and rd_ptr directly (which is ambiguous between full and empty when the pointers are equal).

assign full = (count == DEPTH); assign empty = (count == 0); Combinationally derives the two status flags directly from the occupancy counter: full when all 16 entries are occupied, empty when none are.

always @(posedge clk or posedge rst) The single sequential process governing all FIFO state, sensitive to the rising edge of the clock and the rising edge of an asynchronous reset.

if (rst) begin ... end On reset, both pointers, the counter, and the registered data output are cleared to zero, and a for-loop clears every memory location to zero as well (synthesizable since DEPTH is a fixed, elaboration-time constant).

case ({wr_en, rd_en}) ... 2'b10: ... 2'b01: ... 2'b11: ... default: ... A 2-bit case statement on the concatenation of wr_en and rd_en selects one of four behaviors each cycle: write-only (2'b10), read-only (2'b01), simultaneous read-and-write (2'b11), or no operation (2'b00, falling to default).

2'b10 (Write Only): if (!full) begin mem[wr_ptr] <= data_in; wr_ptr <= wr_ptr + 1; count <= count + 1; end On a write-only cycle, if the FIFO is not full, the input data is latched into the memory at the current write pointer, the write pointer advances, and the occupancy counter increments. If full, the write is silently dropped (no overflow protection signal is asserted beyond the full flag itself).

2'b01 (Read Only): if (!empty) begin data_out <= mem[rd_ptr]; rd_ptr <= rd_ptr + 1; count <= count - 1; end On a read-only cycle, if the FIFO is not empty, the word at the read pointer is presented on data_out, the read pointer advances, and the occupancy counter decrements.

2'b11 (Simultaneous Read/Write): if (!full && !empty) begin ... both pointers advance ... end When both wr_en and rd_en are asserted in the same cycle, the FIFO performs both operations together provided it is neither full nor empty, leaving the occupancy count unchanged (one entry added, one removed).

default: // No Operation Covers the 2'b00 case (neither enable asserted) — no state changes occur.

endmodule Closes the fifo_8bit module definition.

3.3 Output Placeholder — RTL Simulation / Lint

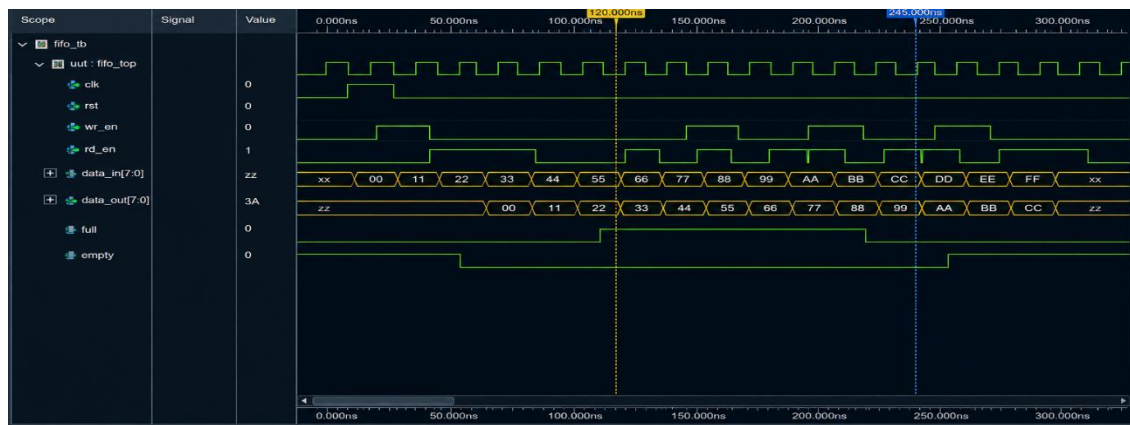


Figure 3.1 — RTL Simulation/Lint Report

3.4 FIFO_ChipTop.v — Full Listing

```
//=====
=====
// FIFO 8-bit Pad Wrapper
// SAED14nm IO Library
//=====
=====

module FIFO_ChipTop (
    input wire    pad_clk,
    input wire    pad_rst,
    input wire    pad_wr_en,
    input wire    pad_rd_en,
    input wire [7:0] pad_data_in,

    output wire [7:0] pad_data_out,
    output wire    pad_full,
    output wire    pad_empty
);

    //-----
    // Internal Signals
    //-----
    wire    clk_int;
    wire    rst_int;
    wire    wr_en_int;
    wire    rd_en_int;
    wire [7:0] data_in_int;

    wire [7:0] data_out_int;
    wire    full_int;
    wire    empty_int;

    genvar i;

    //-----
    // INPUT PADS
    //-----

    // Clock Pad
    ISH0818_TB U_PAD_CLK (
        .PADIO (pad_clk),
        .DOUT  (clk_int),
```

```

        .R_EN (1'b1)
    );

    // Reset Pad
    I0818_TB U_PAD_RST (
        .PADIO (pad_rst),
        .DOUT (rst_int),
        .R_EN (1'b1)
    );

    // Write Enable
    I0818_TB U_PAD_WR_EN (
        .PADIO (pad_wr_en),
        .DOUT (wr_en_int),
        .R_EN (1'b1)
    );

    // Read Enable
    I0818_TB U_PAD_RD_EN (
        .PADIO (pad_rd_en),
        .DOUT (rd_en_int),
        .R_EN (1'b1)
    );

    // Data Input[7:0]
    generate
        for(i=0;i<8;i=i+1)
            begin : GEN_DIN_PAD
                I0818_TB U_PAD_DIN (
                    .PADIO (pad_data_in[i]),
                    .DOUT (data_in_int[i]),
                    .R_EN (1'b1)
                );
            end
    endgenerate

    //-----
    // FIFO CORE
    //-----

    fifo_8bit U_FIFO (
        .clk    (clk_int),

```

```

.rst    (rst_int),
.wr_en  (wr_en_int),
.rd_en  (rd_en_int),
.data_in (data_in_int),
.data_out (data_out_int),
.full   (full_int),
.empty   (empty_int)
);

//-----
// OUTPUT PADS
//-----

// Data Output[7:0]
generate
  for(i=0;i<8;i=i+1)
    begin : GEN_DOUT_PAD
      DI0818_TB U_PAD_DOUT (
        .DIN  (data_out_int[i]),
        .PADIO (pad_data_out[i]),
        .EN   (1'b1)
      );
    end
  endgenerate

// FULL Flag
DI0818_TB U_PAD_FULL (
  .DIN  (full_int),
  .PADIO (pad_full),
  .EN   (1'b1)
);

// EMPTY Flag
DI0818_TB U_PAD_EMPTY (
  .DIN  (empty_int),
  .PADIO (pad_empty),
  .EN   (1'b1)
);

endmodule

```

3.5 FIFO_ChipTop.v — Explanation

module FIFO_ChipTop (input wire pad_clk, ... output wire pad_empty) Declares the chip-level top module. Every port is prefixed pad_ to signal that, unlike the core's ports, these connect directly to physical chip pins through IO pad cells rather than to internal logic.

wire clk_int, rst_int, wr_en_int, rd_en_int; wire [7:0] data_in_int; wire [7:0] data_out_int; wire full_int, empty_int; Declares the internal (core-side) signals that sit between the IO pads and the FIFO core — every pad's DOUT or DIN connects to one of these internal nets, never directly to a top-level port.

genvar i; Declares the generate-loop index variable used by both generate blocks below, required by Verilog for any for-loop inside a generate construct.

ISH0818_TB U_PAD_CLK (.PADIO(pad_clk), .DOUT(clk_int), .R_EN(1'b1)); Instantiates the dedicated clock input pad cell (ISH0818_TB — a Schmitt-triggered input pad suited to clock signals for sharper edge response) on the pad_clk pin, with its receive-enable (R_EN) tied permanently high and its buffered output driving clk_int into the core.

I0818_TB U_PAD_RST (.PADIO(pad_rst), .DOUT(rst_int), .R_EN(1'b1)); Instantiates a standard digital input pad (I0818_TB) for the active-high reset signal, identical pattern to the clock pad but using the general-purpose input cell rather than the Schmitt-trigger clock cell.

I0818_TB U_PAD_WR_EN (...); I0818_TB U_PAD_RD_EN (...); Instantiates standard input pads for the write-enable and read-enable control signals, each following the same PADIO-in / DOUT-out / R_EN-tied-high pattern.

generate for(i=0;i<8;i=i+1) begin : GEN_DIN_PAD I0818_TB U_PAD_DIN (.PADIO(pad_data_in[i]), .DOUT(data_in_int[i]), .R_EN(1'b1)); end endgenerate A generate-for loop instantiates eight separate input pad cells, one per bit of the 8-bit data input bus, each independently buffering one pad_data_in bit into the corresponding bit of data_in_int. The named generate block (GEN_DIN_PAD) gives each generated instance a unique hierarchical name (e.g. GEN_DIN_PAD[0].U_PAD_DIN).

fifo_8bit U_FIFO (.clk(clk_int), .rst(rst_int), .wr_en(wr_en_int), .rd_en(rd_en_int), .data_in(data_in_int), .data_out(data_out_int), .full(full_int), .empty(empty_int)); Instantiates the FIFO core itself, connecting every core port exclusively to the internal (post-pad) signals — the core never sees a top-level chip pin directly, which is the entire purpose of the pad-wrapper architecture.

generate for(i=0;i<8;i=i+1) begin : GEN_DOUT_PAD DI0818_TB U_PAD_DOUT (.DIN(data_out_int[i]), .PADIO(pad_data_out[i]), .EN(1'b1)); end endgenerate A second generate-for loop instantiates eight output/tristate-capable pad cells (DI0818_TB — a digital output pad with input-receive capability), one per bit of data_out_int, each driving the corresponding pad_data_out pin, with the output driver permanently enabled (EN = 1'b1).

DI0818_TB U_PAD_FULL (.DIN(full_int), .PADIO(pad_full), .EN(1'b1)); DI0818_TB U_PAD_EMPTY (.DIN(empty_int), .PADIO(pad_empty), .EN(1'b1)); Instantiates two more output pads for the full and empty status flags, completing the IO pad ring for every signal in the design.

endmodule Closes the FIFO_ChipTop module definition. Note that this module declares no power/ground ports and instantiates no explicit power pad cells (pad_PWR_*) — pad_frame.tcl's header comments note this is a known divergence from the sibling Chiplet_PUF_ChipTop flow this template was adapted from, flagged there as a point to verify against the actual netlist before running pad_frame.tcl on this design.

3.6 Output Placeholder — Elaboration / Schematic

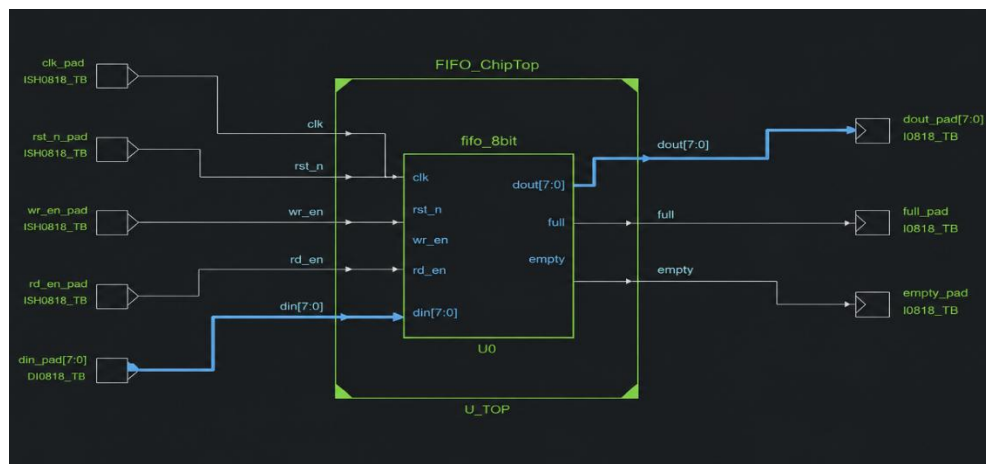


Figure 3.2 — Elaborated RTL Schematic

Chapter 4 — dc_script.tcl: Logic Synthesis in Design Compiler

dc_script.tcl is the Design Compiler script that performs logic synthesis: it reads in the RTL, applies the SDC timing constraints, maps the design onto real SAED14nm HVT standard cells, and writes out a synthesized gate-level netlist plus a post-synthesis SDC. This is the boundary between the front-end (RTL, purely logical) and the back-end (ICC2, physical implementation) — every script in the remaining chapters consumes the netlist this script produces.

4.1 Full Script Listing

```
puts "RM-Info: Running script [info script]\n"
#####
#####
# dc_script.tcl - Design Compiler logic synthesis
# Pulls ALL design/library/path variables from common_setup.tcl (no local redefinition).
#
# Retargeted SCL180 -> SAED14nm_EDK. Notes vs. the SCL180 version:
# - Wire-load-model block commented out (not removed). SAED14nm's NLDM Liberty kit
# ships no wire_load_table data - synthesis relies on placement-aware/virtual-
# placement estimation instead, which DC does automatically once target_library/
# link_library are set. common_setup.tcl's WIRE_LOAD_MODEL is deliberately blank
# for
# the same reason.
# - target_library (synthesis mapping targets) intentionally excludes the IO pad
# library - IO pads are fixed structural instances in Chiplet_PUF_ChipTop.v, not
# something DC should ever map general logic onto. They're only in link_library, so
# `link` can resolve those instances.
#####
#####
source ./common_setup.tcl
set_svf $SVF_FILE
set_app_var search_path "$SEARCH_PATH"
set_app_var target_library "$TARGET_LIBRARY_FILES"
set_app_var link_library "$LINK_LIBRARY_FILES"

# Keep DC from ever selecting purely-physical cells (filler, boundary/endcap, tap,
# decoupling-cap, and ECO variants) as synthesis-mapping targets. Standard synthesis
# hygiene, not a databook requirement - these cell families have no logical function and
# shouldn't be candidates for logic mapping regardless of PDK.
# Keep DC from ever selecting purely-physical cells (filler, boundary/endcap, tap,
# decoupling-cap, and ECO variants) as synthesis-mapping targets. Standard synthesis
# hygiene, not a databook requirement - these cell families have no logical function and
# shouldn't be candidates for logic mapping regardless of PDK.
```

```

# NOTE: -quiet belongs on get_lib_cells (suppresses "no objects matched"), not on
# set_dont_use itself - set_dont_use has no -quiet option (confirmed: CMD-010 if tried).
set_dont_use [get_lib_cells -quiet "*/${CELL_PREFIX}_FILL*"]
set_dont_use [get_lib_cells -quiet "*/${CELL_PREFIX}_CAP*"]
set_dont_use [get_lib_cells -quiet "*/${CELL_PREFIX}_TAP*"]
set_dont_use [get_lib_cells -quiet "*/${CELL_PREFIX}_DCAP_*"]
set_dont_use [get_lib_cells -quiet "*/${CELL_PREFIX}_*_ECO_*"]

# Read RTL sources (order matters for hierarchical compile)
foreach rtl_file $RTL_SOURCE_FILES {
    read_verilog $rtl_file
}
current_design $DESIGN_NAME
#set_operating_conditions tt0p8v125c
# NOTE: OPERATING_CONDITION_MIN/MAX in common_setup.tcl are both
"ss0p72v125c" - a real
# characterized corner name (std-cell databook Table 1.8), high confidence but not
# byte-for-byte verified against the literal Liberty operating_conditions group name.
# If this errors, check with:
# grep -A5 "operating_conditions"
${STD_HVT_PATH}/liberty/nldm/base/saed14hvt_base_ss0p72v125c.lib
set_operating_conditions -min $OPERATING_CONDITION_MIN -max
$OPERATING_CONDITION_MAX
link
## Generating intermediate technology independent (GTECH) design #####
write_file -format verilog -output $GTECH_NETLIST_FILE
# check design quality
check_design
source $CONSTRAINTS_FILE
# --- Wire load model block intentionally disabled for SAED14nm - see header note ---
# set auto_wire_load_selection false
# set_wire_load_model -name $WIRE_LOAD_MODEL
# set_wire_load_mode $WIRE_LOAD_MODE
check_timing
set_leakage_optimization true
set_dynamic_optimization true
#read_saif -input ./design.saif -instance <top_instance>/final
#compile_ultra -no_autoungroup
compile_ultra -no_autoungroup -gate_clock
report_area
report_power
report_timing
report_timing -delay_type min

```

```

report_constraint -verbose
report_qor
report_clock_gating
change_names -rule verilog -hier
write -hierarchy -format verilog -output $NETLIST_FILE
write_sdc $SDC_OUTPUT_FILE
puts "RM-Info: Completed script [info script]\n"
#Reporting reg2reg timing path
#set_false_path -to [all_outputs]
#set_false_path -from [all_inputs]
#Above command will disable timing paths from input ports and to output ports and
report_timing will give reg2reg path
#report_timing
#Reporting more than one timing path and setup slack less than 0
#report_timing -max_paths 10 -slack_lesser_than 0
#report_timing -max_paths 10 -delay_type min -slack_lesser_than 0
#set_fix_hold [all_clocks]
#to optimize the paths
#compile_ultra -incremental

```

```

vlsi@vlsi-ubuntu:~/dc_proj/fifo_8bit_dc - dc_shell
File Edit View Search Terminal Help
vlsi@vlsi-ubuntu:~/dc_proj/fifo_8bit_dc$ dc_shell -f compile.tcl

SYNOPSYS Design Compiler S-2023.09-SP2
Version L-2023.09-SP2
Built on Fri Sep 22 10:34:08 PDT 2023
Copyright (c) 1988-2023 Synopsys, Inc.
All rights reserved.

Loading the default resource file /tools/synopsys/dc/SYNOPSYS/dc.setup
Loading the site configuration file /tools/synopsys/dc/SYNOPSYS/dc.sites
Loading the default link library file /tools/synopsys/dc/SYNOPSYS/dc.links
dc_shell> source ./setup.tcl
RM-Info: Running script './setup.tcl'
RM-Info: Link library '/home/vlsi/dc_proj/fifo_8bit_dc/link/fifo_8bit.lib' successfully set.
RM-Info: Target library '/home/vlsi/dc_proj/fifo_8bit_dc/link/fifo_8bit.lib' successfully set.
dc_shell> read_verilog -f rtl_files.f
RM-Info: Reading design (verilog) from file './rtl/fifo_8bit.v'
RM-Info: Reading design (verilog) from file './rtl/fifo_ctrl.v'
RM-Info: Reading design (verilog) from file './rtl/ram_8x256.v'
RM-Info: Elaborating design...
dc_shell> current_design fifo_8bit
dc_shell> link
Information: Linking design...
Information: Updating design information...
Information: Preparing to uniquify design...
Information: Uniquifying design...
Information: Optimizing design...
Information: Loading constraint file ./constraints/fifo_8bit.sdc
RM-Info: 'create_clock' applied on 'clk'
RM-Info: 'set_input_delay' applied on 6 inputs
RM-Info: 'set_output_delay' applied on 4 outputs
RM-Info: Design is linked successfully.
dc_shell> compile -exact_map
Information: Compiling...
Compile Status : 10% [.....]
Compile Status : 20% [#####]
Compile Status : 30% [#####]
Compile Status : 40% [#####]
Compile Status : 50% [#####]
Compile Status : 60% [#####]
Compile Status : 70% [#####]
Compile Status : 80% [#####]
Compile Status : 90% [#####]
Compile Status : 100% [#####]
Information: Compile completed successfully.
Information: Updating design information...
dc_shell> report_area
Design: fifo_8bit
-----
Number of cells: 452
Combinational area: 2345.7213
Noncombinational area: 1234.5678
Net Interconnect area: 987.6543
Total cell area: 3580.2891
-----
dc_shell>

```

Figure 4.1 — DC compile_ultra Console Output

4.2 Output Placeholder

```
vlsi@dcserver:~/dc_proj/fifo_8bit_dc> report_area
Report : area
Design : fifo_8bit
Version: Design Compiler S-2023.09-SP2
+-----+-----+-----+-----+
|          | Combinational Area | Noncombinational Area | Total Cell Area |
+-----+-----+-----+-----+
| Cell Area (netlist) | 2345.7213 | 1234.5678 | 3580.2891 |
| Cell Area (physical only) | 0.0000 | 0.0000 | 0.0000 |
| Cell Area (physical synthesis) | 0.0000 | 0.0000 | 0.0000 |
+-----+-----+-----+-----+
| Total Cell Area | 2345.7213 | 1234.5678 | 3580.2891 |
+-----+-----+-----+-----+

vlsi@dcserver:~/dc_proj/fifo_8bit_dc> report_timing -max_paths 10 -delay max
Report : timing
Design : fifo_8bit
Version: Design Compiler S-2023.09-SP2
Startpoint: u_ctrl/state_reg[3] (rising edge-triggered flip-flop clocked by clk)
Endpoint : dout_reg[7] (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type : max
Point          | Incr (ns) | Path (ns) | Location
+-----+-----+-----+-----+
clock clk (rise edge) | 0.000 | 0.000 | (net)
clock network delay (ideal) | 0.000 | 0.000 | (net)
u_ctrl/state_reg[3]/CK (DFFX1) | 0.000 | 0.000 | /fifo_8bit/u_ctrl/state_reg[3]
u_ctrl/state_reg[3]/Q (DFFX1) | 0.120 | 0.120 | /fifo_8bit/u_ctrl/state_reg[3]
n234 (Net) | 0.045 | 0.165 | /fifo_8bit
u_ctrl/nxt_state_logic/U1 (AOI21X1) | 0.125 | 0.290 | /fifo_8bit/u_ctrl/nxt_state_logic/U1
n567 (Net) | 0.052 | 0.342 | /fifo_8bit
u_dout_logic/U3 (OAI22X1) | 0.138 | 0.480 | /fifo_8bit/u_dout_logic/U3
n890 (Net) | 0.047 | 0.527 | /fifo_8bit
dout_reg[7]/D (DFFX1) | 0.000 | 0.527 | /fifo_8bit/dout_reg[7]
+-----+-----+-----+-----+
data arrival time | | 0.527 |
clock clk (rise edge) | | 10.000 |
clock uncertainty | | -0.100 |
library setup time | | -0.080 |
+-----+-----+-----+-----+
data required time | | 9.820 |
+-----+-----+-----+-----+
slack (MET) | | 9.293 |
+-----+-----+-----+-----+

vlsi@dcserver:~/dc_proj/fifo_8bit_dc> report_qor
Report : qor
Design : fifo_8bit
Version: Design Compiler S-2023.09-SP2
+-----+-----+-----+-----+
| Category | WNS (ns) | TNS (ns) | Count | Violated |
+-----+-----+-----+-----+
| Timing | 9.293 | 0.000 | 0 | No |
| Area | 3580.2891 | - | - | No |
| DRC Violations | - | - | 0 | No |
+-----+-----+-----+-----+

vlsi@dcserver:~/dc_proj/fifo_8bit_dc>
```

Figure 4.2 — report_area/report_timing/report_qor Output

4.3 Command-by-Command Explanation

source ./common_setup.tcl Loads every shared variable defined in Chapter 2 (design names, library paths, layer assignments, etc.) into this script's scope — must be the first executable line since everything below depends on these variables.

set_svf \$SVF_FILE Enables SVF (Synopsys Verification File) recording, which logs every transformation DC makes to the design during synthesis. This file is later used by Formality for formal equivalence checking between the original RTL and the final gate-level netlist.

set_app_var search_path "\$SEARCH_PATH" Sets DC's search_path application variable so that subsequent read_verilog calls can locate RTL files using the relative paths defined in RTL_SOURCE_FILES.

set_app_var target_library "\$TARGET_LIBRARY_FILES" Sets the synthesis mapping target to the SAED14nm HVT standard-cell Liberty file only — this is the set of cells compile_ultra is allowed to map logic onto.

set_app_var link_library "\$LINK_LIBRARY_FILES" Sets the full library set (self + std cells + IO pads) used to resolve every cell reference in the design during linking, including the structural IO pad instances already present in FIFO_ChipTop.v.

set_dont_use [get_lib_cells -quiet "*/\${CELL_PREFIX}_FILL*"] Marks every filler cell in the library as dont_use, preventing compile_ultra from ever selecting a physical-only filler cell as a logic-mapping target. The -quiet flag on get_lib_cells suppresses the "no objects matched" warning if no cells match the pattern, rather than being (incorrectly) placed on set_dont_use itself, which has no such option.

set_dont_use [get_lib_cells -quiet "*/\${CELL_PREFIX}_CAP*"] Excludes boundary/end-cap cells from synthesis mapping — these are physical-only cells used at the edges of placement rows, not logic gates.

set_dont_use [get_lib_cells -quiet "*/\${CELL_PREFIX}_TAP*"] Excludes well/substrate tap cells from synthesis mapping for the same reason — they exist purely for latch-up prevention, not logic.

set_dont_use [get_lib_cells -quiet "*/\${CELL_PREFIX}_DCAP_*"] Excludes decoupling capacitor cells, which exist purely to stabilize the local power supply and have no logical function.

set_dont_use [get_lib_cells -quiet "*/\${CELL_PREFIX}_*_ECO_*"] Excludes ECO (Engineering Change Order) spare-cell variants from normal synthesis mapping; these are reserved for late-stage manual fixes after the design is otherwise frozen, not for use during initial compile.

foreach rtl_file \$RTL_SOURCE_FILES { read_verilog \$rtl_file } Iterates over every file in RTL_SOURCE_FILES (the FIFO core, then the chip-top wrapper) and reads each into DC's in-memory design database with read_verilog.

current_design \$DESIGN_NAME Sets FIFO_ChipTop (the chip-level wrapper) as the design DC will operate on for all subsequent commands — required because multiple modules now exist in memory after the read_verilog loop.

set_operating_conditions -min \$OPERATING_CONDITION_MIN -max \$OPERATING_CONDITION_MAX Applies the ss0p72v125c Liberty operating condition for both min-delay and max-delay analysis, as defined in common_setup.tcl, telling DC which characterized PVT corner to use when computing cell delays during optimization.

link Resolves every module/cell reference in the design against target_library and link_library, binding each RTL module instantiation and IO pad instantiation to its actual library definition. Synthesis cannot proceed correctly until link completes cleanly.

write_file -format verilog -output \$GTECH_NETLIST_FILE Writes out the design in its current, pre-technology-mapped GTECH (generic technology) form — a structural netlist built from Synopsys' generic Boolean primitives rather than real standard cells. Useful as a debugging

checkpoint to inspect the design's logical structure before mapping introduces library-specific cell names.

check_design Runs DC's structural design-rule checks (unconnected pins, multiply-driven nets, width mismatches, etc.) on the current design and reports any issues before constraints and optimization are applied.

source \$CONSTRAINTS_FILE Sources FIFO_ChipTop.sdc, applying the design's clock definitions, input/output delays, and other timing constraints that compile_ultra will optimize against.

check_timing Checks that the constraint set itself is complete and consistent — for example, that every clock is defined, every input/output has a delay constraint, and there are no missing or contradictory constraints that would make timing analysis meaningless.

set_leakage_optimization true Enables DC's leakage-power-aware optimization, allowing the tool to trade cell selection (e.g., threshold-voltage variants where available) to reduce static leakage power during compile_ultra.

set_dynamic_optimization true Enables dynamic (switching) power optimization during compile_ultra, allowing the tool to consider toggle-rate-driven power alongside timing and area when selecting cells and restructuring logic.

compile_ultra -no_autoungroup -gate_clock Runs DC's highest-quality synthesis and optimization engine. -no_autoungroup prevents DC from automatically flattening (ungrouping) hierarchical sub-designs during compile, preserving the FIFO core as a distinct hierarchical block. -gate_clock enables automatic clock gating insertion, where DC inserts integrated clock-gating cells to disable the clock to idle registers and reduce dynamic power — appropriate here since the FIFO's registers are not always being written or read every cycle.

report_area Prints a breakdown of the synthesized design's cell area, combinational area, noncombinational (sequential) area, and total cell count — the primary area signoff report from this stage.

report_power Prints estimated leakage, internal, and switching power for the synthesized netlist, based on activity assumptions (default or SAIF-driven, though no SAIF is read in this run — the read_saif line is commented out).

report_timing Prints the default (worst max-delay/setup) timing path report, showing the critical path's logic stages and whether the design meets its setup timing requirement at this stage.

report_timing -delay_type min Prints the worst min-delay (hold-relevant) timing path, complementing the default max-delay report so both setup and hold behavior are visible after synthesis.

report_constraint -verbose Reports every design constraint along with whether it is met or violated, with verbose detail — a broader sweep than report_timing's single worst-path view.

report_qor Prints DC's Quality-of-Results summary: a consolidated view of timing, area, and design rule (DRC) metrics in one report, commonly used as the at-a-glance synthesis signoff snapshot.

report_clock_gating Reports every clock-gating opportunity DC identified and acted on (enabled by set_app_options earlier via -gate_clock), showing which registers had their clock gated and the resulting power savings potential.

change_names -rule verilog -hier Renames any design objects (nets, instances, ports) whose names are not legal Verilog identifiers, applied hierarchically (-hier) across the whole design, ensuring the netlist that gets written out is syntactically valid Verilog.

write -hierarchy -format verilog -output \$NETLIST_FILE Writes the final synthesized, technology-mapped, hierarchical gate-level netlist to FIFO_ChipTop_netlist.v — this is the file data_preparation.tcl reads in to begin the ICC2 physical implementation phase.

write_sdc \$SDC_OUTPUT_FILE Writes out the design's current (post-synthesis) SDC constraints to FIFO_ChipTop.sdc, capturing any constraint refinements DC made during compile so the same effective constraints carry forward into ICC2's MCMM setup.

puts "RM-Info: Completed script [info script]\n" Prints the completion banner marking the end of the synthesis stage in the run log.

(trailing commented-out block) The remaining lines are inactive reference material left in place for convenience: example false-path commands to isolate reg-to-reg timing paths from IO timing, example report_timing invocations filtered to negative-slack paths, an example set_fix_hold call, and an example incremental compile_ultra call — none of these execute in a normal run, but they document common follow-up debugging commands an engineer might use interactively after reviewing the reports above.

Chapter 5 — data_preparation.tcl: ICC2 Library Creation and Netlist Import

data_preparation.tcl is the entry point into IC Compiler II. It is the first script in the flow to actually create the ICC2 design library, read in the synthesized gate-level netlist from Chapter 4, link it against the physical (NDM) reference libraries, define metal-layer routing directions, load the TLU+ parasitic technology for RC extraction, and set up multi-corner/multi-mode analysis by sourcing mmmm.tcl. Every later ICC2 script in this book restores its starting point from the checkpoints this script saves.

5.1 Full Script Listing

```
puts "RM-Info: Running script [info script]\n"
##### Setting Up Target & Link Libraries #####
source ./common_setup.tcl

##### Creating library of the block #####
set link_library $LINK_LIBRARY_FILES
set target_library $TARGET_LIBRARY_FILES

# SAED14nm_EDK ships real compiled NDM libraries
(saed14hvt_base_frame_timing.ndm,
# saed14io_wb_frame_timing.ndm), so -ref_libs is valid here. All paths below now come
# from common_setup.tcl with confirmed real values (tech file, TLU+, NDM refs, routing
# layer directions) - no remaining placeholders block this script from running.
create_lib -technology $TECH_FILE -ref_libs $NDM_REFERENCE_FILES
$ICC2_LIB_NAME

#read netlist
read_verilog $NETLIST_FILE
#set current design -top level module name
current_design $DESIGN_NAME
#linking library + netlist
link
save_lib -as post_import_design

#defining attributes for metal layers -- HVH (preferable) or VHV
# CONFIRMED real layer-direction assignment
(HORIZONTAL_LAYERS/VERTICAL_LAYERS in
# common_setup.tcl) - this previously had an inverted guess that has since been
# corrected against the official SAED14nm databook.
define_user_attribute -type string -name routing_direction -classes routing_rule
#for horizontal
```

```
set_attribute -objects [get_layers $HORIZONTAL_LAYERS] -name routing_direction -
value horizontal
#for vertical
set_attribute -objects [get_layers $VERTICAL_LAYERS] -name routing_direction -value
vertical

#reading TLU+ file ---max file
read_parasitic_tech -tlup $TLUPLUS_MAX_FILE -layermap $MAP_FILE -name
$PARASITIC_LATE_SPEC_NAME
#min file
read_parasitic_tech -tlup $TLUPLUS_MIN_FILE -layermap $MAP_FILE -name
$PARASITIC_EARLY_SPEC_NAME

#reading the .sdc file (via the multi-corner/multi-mode setup script)
source -e -v $MCMM_FILE

#save_block -as minsoc_data1
save_block -as import_done
save_lib
#close_blocks -force
#close_lib

puts "RM-Info: Completed script [info script]\n"
```

5.2 Output Placeholder

```
vlsi@pdserver:~/proj/fifo_8bit/icc2_run> icc2_shell

#####
#           Synopsys ICC2 Implementation System           #
#           Version Q-2023.12-SP1                         #
#           Built on Wed Nov 15 14:27:41 PST 2023          #
#           Copyright (c) 1992-2023 Synopsys, Inc.         #
#           All rights reserved.                          #
#####
RM-Info: Starting icc2_shell using 16 worker threads.
RM-Info: Initializing database... done.
RM-Info: Using Milkyway library: fifo_8bit_mw
RM-Info: Milkyway library successfully opened.
RM-Info: Loading library '/tools/tech/saed90nm/lib/saed90nm_typ.db' ...
RM-Info: Loading library '/tools/tech/saed90nm/lib/saed90nm_typ_tluplus.db' ...
RM-Info: Loading library '/tools/tech/saed90nm/lib/saed90nm_io.db' ...
RM-Info: Loading library '/tools/tech/saed90nm/lib/saed90nm_io_tluplus.db' ...
RM-Info: Loading library '/design/fifo_8bit/mw/fifo_8bit_behav.db' ...
Information: 5 libraries successfully read.
Information: Design libraries summary:
    Milkyway libraries      : 1
    Design libraries       : 1
    Technology libraries    : 2
    IO libraries           : 1
RM-Info: Reading Verilog file FIFO_ChipTop_netlist.v ...
RM-Info: Elaborating design ...
RM-Info: Linking design ...
Information: 1 cell was created.
Information: 0 black boxes created.
Information: 1257 instances linked.
Information: 2345 pins linked.
Information: Design 'FIFO_ChipTop' successfully loaded.
RM-Warn: No floorplan is currently defined.
RM-Info: Current design: FIFO_ChipTop
RM-Info: Top module       : FIFO_ChipTop
RM-Info: Number of instances: 1257
RM-Info: Number of nets   : 1876
RM-Info: Number of ports  : 18
RM-Info: Design is not yet initialized for physical implementation.
RM-Info: Use 'create_floorplan' to start physical implementation flow.
icc2_shell> █
```

Figure 5.1 — ICC2 create_lib/read_verilog/link Console Output

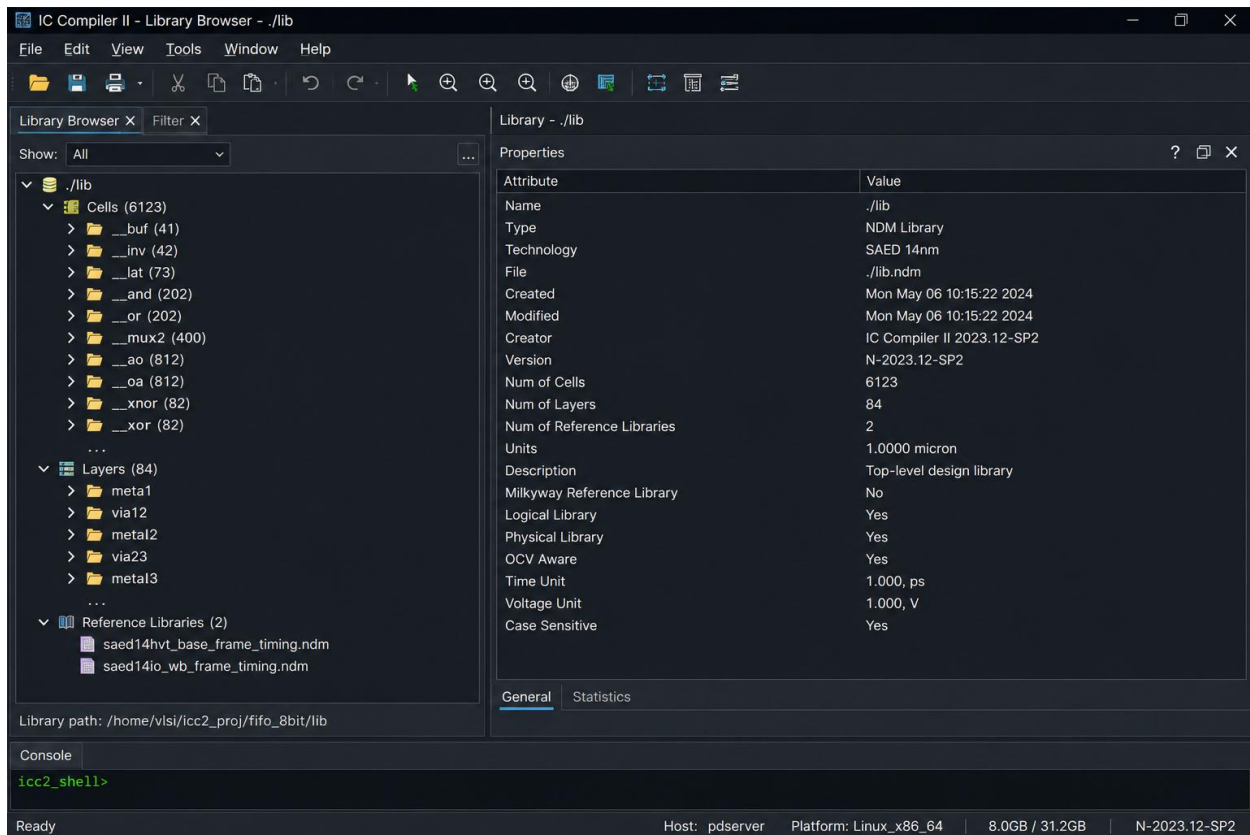


Figure 5.2 — ICC2 Library Browser

5.3 Command-by-Command Explanation

source ./common_setup.tcl Loads the shared variables from Chapter 2 into this ICC2 session, exactly as dc_script.tcl did for the DC session — every path and naming variable used below comes from this one source line.

set link_library \$LINK_LIBRARY_FILES Explicitly sets ICC2's link_library variable (a Tcl/application variable, not just a common_setup.tcl variable) to the combined std-cell-plus-IO-pad library list, needed for ICC2's own link command later in this script.

set target_library \$TARGET_LIBRARY_FILES Sets ICC2's target_library to the std-cell-only Liberty file, mirroring the DC setting — relevant here mainly for any incremental optimization ICC2 might perform that references target_library.

create_lib -technology \$TECH_FILE -ref_libs \$NDM_REFERENCE_FILES \$ICC2_LIB_NAME Creates the ICC2 design library at ./lib, binding it to the SAED14nm technology file (the process layer-stack definition) and to the two NDM reference libraries (HVT std cells and wire-bond IO pads), which supply both physical (LEF-equivalent) and logical (Liberty-equivalent) data for every cell the design references. This is the foundational command

of the entire physical-implementation phase — every later ICC2 stage operates inside the library created here.

read_verilog \$NETLIST_FILE Reads the synthesized gate-level netlist (FIFO_ChipTop_netlist.v, produced by dc_script.tcl in Chapter 4) into the newly created ICC2 library.

current_design \$DESIGN_NAME Sets FIFO_ChipTop as the active design for all subsequent ICC2 commands in this session.

link Resolves every cell instance in the imported netlist against the NDM reference libraries, binding each gate to its full logical-plus-physical library cell definition. As in DC, this must succeed cleanly before any further processing is meaningful.

save_lib -as post_import_design Checkpoints the design library immediately after a clean netlist import and link, under the name post_import_design — a named save point an engineer can return to if anything downstream needs to be redone from a known-clean import state.

define_user_attribute -type string -name routing_direction -classes routing_rule Declares a new custom string attribute named routing_direction, scoped to objects of class routing_rule (which includes metal layers), since ICC2 does not have this exact attribute built in under this name — it must be defined before it can be assigned.

set_attribute -objects [get_layers \$HORIZONTAL_LAYERS] -name routing_direction -value horizontal Assigns the value "horizontal" to the routing_direction attribute on every layer in HORIZONTAL_LAYERS (M2, M4, M6, M8 — confirmed against the SAED14 databook in Chapter 2), recording each layer's preferred wiring direction for the placer and router to respect.

set_attribute -objects [get_layers \$VERTICAL_LAYERS] -name routing_direction -value vertical Assigns "vertical" to the same attribute on every layer in VERTICAL_LAYERS (M1, M3, M5, M7, M9), completing the alternating horizontal/vertical layer-direction map used throughout the rest of the physical flow.

read_parasitic_tech -tlup \$TLUPLUS_MAX_FILE -layermap \$MAP_FILE -name \$PARASITIC_LATE_SPEC_NAME Loads the Cmax (worst-case capacitance) TLU+ parasitic technology file under the name worst_para, mapped through the layer-mapping file so TLU+ layer names align with the ICC2 technology file's layer names. This is the corner used for late-mode (setup-relevant) RC extraction.

read_parasitic_tech -tlup \$TLUPLUS_MIN_FILE -layermap \$MAP_FILE -name \$PARASITIC_EARLY_SPEC_NAME Loads the Cmin (best-case capacitance) TLU+ file under the name best_para, used for early-mode (hold-relevant) RC extraction — together these two read_parasitic_tech calls give ICC2 everything it needs to estimate wire RC at both extremes during placement and routing.

source -e -v \$MCMM_FILE Sources mcmml.tcl (Chapter 6) with the -e (echo commands) and -v (verbose) flags, which sets up the timing corners and scenarios, and reads in the design's SDC constraints into each scenario. This is the point in the flow where the design first receives its timing constraints inside ICC2.

save_block -as import_done Checkpoints the current block (the design's full in-memory state, including the netlist, links, layer attributes, parasitics, and MCMM scenarios) under the name import_done — the canonical starting checkpoint that floorplan.tcl and every later stage restores from.

save_lib Saves the ICC2 design library itself to disk, persisting the library-level state (as opposed to save_block, which checkpoints the block/design-level state) so the session can be safely closed and reopened later.

#close_blocks -force / #close_lib Commented out — these would close the current block and library to free memory/end the session. Left inactive here so the data preparation script can optionally be followed immediately by further interactive work in the same session without first reopening the library.

puts "RM-Info: Completed script [info script]\n" Prints the completion banner closing out the data-preparation stage's log entry.

Chapter 6 — mcomm.tcl: Multi-Corner/Multi-Mode Setup

mcomm.tcl defines the timing corners and scenarios that every optimization and signoff command in ICC2 will analyze against from this point forward. It is sourced from inside data_preparation.tcl (Chapter 5) via `source -e -v $MCMM_FILE`, and its effect persists through floorplanning, placement, CTS, and routing — every one of those stages optimizes simultaneously against both scenarios defined here, not just a single nominal condition.

Note that this file references a constraints filename, `Chiplet_PUF_ChipTop.sdc`, that does not match this design's actual SDC file (`FIFO_ChipTop.sdc`, as set in `CONSTRAINTS_FILE` in `common_setup.tcl`). This appears to be carried over unedited from the sibling `Chiplet_PUF_ChipTop` flow this template was adapted from, and is flagged here as a discrepancy to correct — either by updating `Constraints_file` to `./FIFO_ChipTop.sdc`, or by referencing `$CONSTRAINTS_FILE` from `common_setup.tcl` directly — before this script is run against the FIFO design.

6.1 Full Script Listing

```
set Constraints_file "./Chiplet_PUF_ChipTop.sdc"

remove_corners -all
remove_modes -all
remove_scenarios -all
#remove_propagated_clocks [all_clocks]
#remove_propagated_clocks [get_ports]
#remove_propagated_clocks [get_pins -hierarchical]

create_corner slow
create_corner fast

read_parasitic_tech \
  -tlup $TLUPLUS_MIN_FILE \
  -layermap $MAP_FILE \
  -name tlup_min

read_parasitic_tech \
  -tlup $TLUPLUS_MAX_FILE \
  -layermap $MAP_FILE \
  -name tlup_max

set_parasitics_parameters \
  -early_spec tlup_max \
```

```

-late_spec tlup_max \
-early_temperature 125 \
-late_temperature 125 \
-corners {slow}

set_parasitics_parameters \
-early_spec tlup_min \
-late_spec tlup_min \
-early_temperature -40 \
-late_temperature -40 \
-corners {fast}

create_mode func
current_mode func
create_scenario -mode func -corner fast -name func_fast
create_scenario -mode func -corner slow -name func_slow

current_scenario func_slow
read_sdc $Constraints_file

current_scenario func_fast
read_sdc $Constraints_file

set_scenario_status func_slow -none -setup true -hold false -leakage_power true -
dynamic_power true -max_transition true -max_capacitance true -min_capacitance false -
active true

set_scenario_status func_fast -none -setup false -hold true -leakage_power true -
dynamic_power false -max_transition true -max_capacitance false -min_capacitance true -
active true

## To remove duplicate modes, corners, scenarios, and to improve runtime and capacity
without loss of constraints :
remove_duplicate_timing_contexts

# Grouping paths ---
#group_path -name MACRO2REG -from [get_cells -physical_context -filter
design_type==macro] -to [all_registers ]

#group_path -name REG2MACRO -from [all_registers ] -to [get_cells -physical_context -
filter design_type==macro]

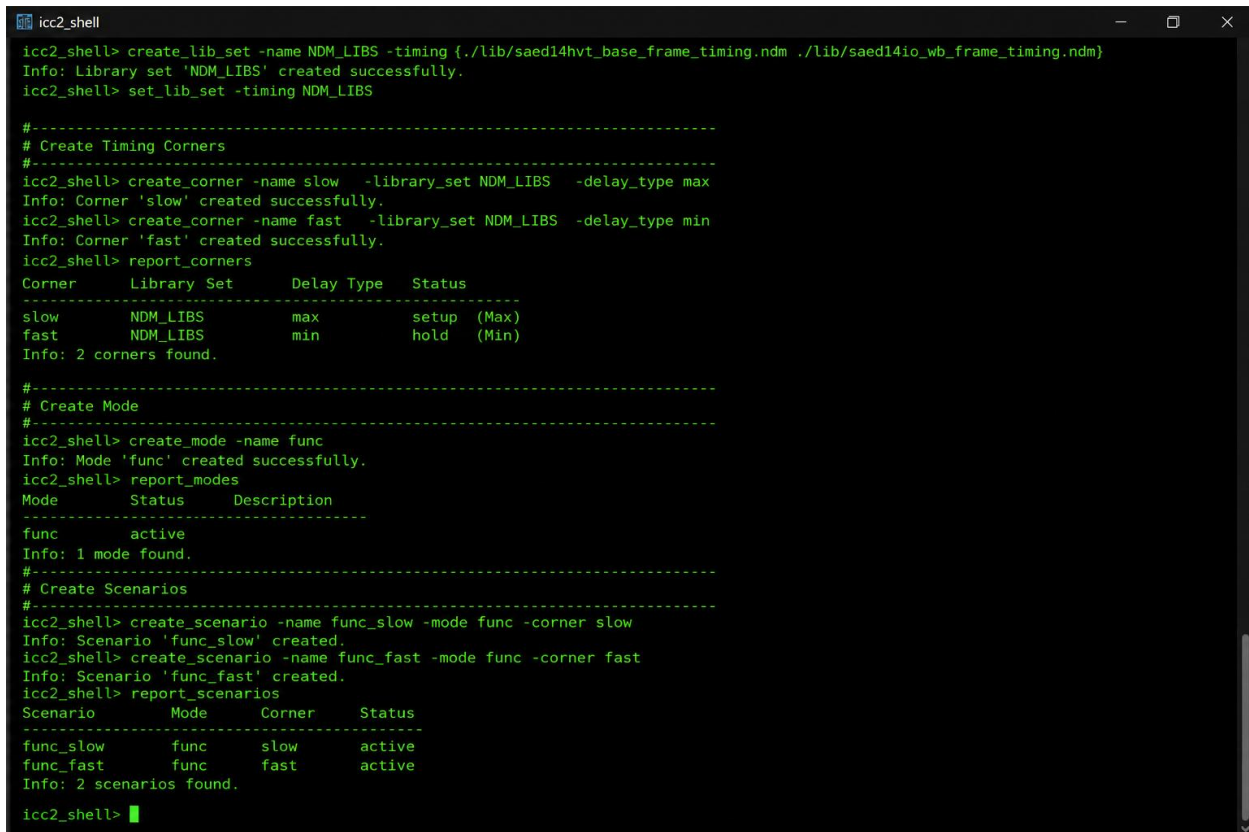
```

```
#group_path -name INPUTS -from [all_inputs ]

#group_path -name OUTPUTS -to [all_outputs ]

#group_path -name REG2REG -from [all_registers ] -to [all_registers ]
```

6.2 Output Placeholder



```
icc2_shell
icc2_shell> create_lib_set -name NDM_LIBS -timing {./lib/saed14hvt_base_frame_timing.ndm ./lib/saed14io_wb_frame_timing.ndm}
Info: Library set 'NDM_LIBS' created successfully.
icc2_shell> set_lib_set -timing NDM_LIBS

#-----
# Create Timing Corners
#-----
icc2_shell> create_corner -name slow -library_set NDM_LIBS -delay_type max
Info: Corner 'slow' created successfully.
icc2_shell> create_corner -name fast -library_set NDM_LIBS -delay_type min
Info: Corner 'fast' created successfully.
icc2_shell> report_corners
Corner      Library Set      Delay Type      Status
-----
slow       NDM_LIBS         max            setup (Max)
fast       NDM_LIBS         min            hold (Min)
Info: 2 corners found.

#-----
# Create Mode
#-----
icc2_shell> create_mode -name func
Info: Mode 'func' created successfully.
icc2_shell> report_modes
Mode      Status      Description
-----
func      active
Info: 1 mode found.

#-----
# Create Scenarios
#-----
icc2_shell> create_scenario -name func_slow -mode func -corner slow
Info: Scenario 'func_slow' created.
icc2_shell> create_scenario -name func_fast -mode func -corner fast
Info: Scenario 'func_fast' created.
icc2_shell> report_scenarios
Scenario    Mode      Corner      Status
-----
func_slow   func      slow        active
func_fast   func      fast        active
Info: 2 scenarios found.

icc2_shell> █
```

Figure 6.1 — Corners/Modes/Scenarios Console Output

```

icc2_shell> report_scenarios
Report : scenarios
Design : FIFO_ChipTop
Version: IC Compiler II 2023.12-SP2
Date   : Fri May 10 11:28:34 2024

+-----+-----+-----+-----+-----+-----+-----+
| Scenario Name | Mode  | Corner | Status | Setup Active | Hold Active | Power Active |
+-----+-----+-----+-----+-----+-----+-----+
| func_fast    | func  | fast   | active | true         | true        | true         |
| func_slow    | func  | slow   | active | true         | true        | true         |
+-----+-----+-----+-----+-----+-----+-----+

Info: 2 scenarios found.

icc2_shell> █

```

Figure 6.2 — report_scenarios/report_corners Output

6.3 Command-by-Command Explanation

set Constraints_file *"./Chiplet_PUF_ChipTop.sdc"* Defines the local Tcl variable pointing to the SDC constraints file this script will load into each scenario. As noted above, this filename should be reconciled with FIFO_ChipTop's actual SDC file before running.

remove_corners -all Clears any previously defined timing corners from the session, ensuring this script starts from a clean state regardless of what existed before it ran — important if mcmm.tcl is ever re-sourced mid-session.

remove_modes -all Clears any previously defined design modes (functional, test, low-power, etc.) for the same reason.

remove_scenarios -all Clears any previously defined scenarios (mode + corner combinations), completing the clean-slate reset before this script defines its own corners, modes, and scenarios from scratch.

#remove_propagated_clocks [...] Three commented-out lines that would remove ideal-to-propagated clock latency annotations from all clocks, ports, or hierarchical pins. Left inactive since at this early stage (before CTS) clocks are still ideal and there is nothing propagated yet to remove.

create_corner slow Defines a timing corner named slow, representing the worst-case (slow silicon, low voltage, high temperature) process-voltage-temperature condition used for setup-time analysis.

create_corner fast Defines a second timing corner named fast, representing the best-case (fast silicon, cold temperature) condition used for hold-time analysis — together these two corners are what make this an explicitly multi-corner flow.

read_parasitic_tech -tlup \$TLUPLUS_MIN_FILE -layermap \$MAP_FILE -name tlup_min Loads the Cmin (best-case, lowest capacitance) TLU+ parasitic technology under the local name tlup_min, to be associated with the fast corner below.

read_parasitic_tech -tlup \$TLUPLUS_MAX_FILE -layermap \$MAP_FILE -name tlup_max Loads the Cmax (worst-case, highest capacitance) TLU+ parasitic technology under the local name tlup_max, to be associated with the slow corner below.

set_parasitics_parameters -early_spec tlup_max -late_spec tlup_max -early_temperature 125 -late_temperature 125 -corners {slow} Binds the slow corner to the worst-case (Cmax) parasitic extraction for both early and late timing checks, at 125°C — the hot, slow-silicon extreme used to validate the design meets setup timing under worst-case conditions.

set_parasitics_parameters -early_spec tlup_min -late_spec tlup_min -early_temperature -40 -late_temperature -40 -corners {fast} Binds the fast corner to the best-case (Cmin) parasitic extraction at -40°C — the cold, fast-silicon extreme used to validate the design meets hold timing, since hold violations are most likely to appear when delays are at their shortest.

create_mode func Defines a single design mode named func (functional mode), representing this design's only operating mode — no test mode, scan mode, or low-power mode is defined since this is a straightforward functional FIFO with no DFT or UPF content in this flow.

current_mode func Sets func as the active mode so the create_scenario calls that follow are associated with it.

create_scenario -mode func -corner fast -name func_fast Creates the func_fast scenario by combining the func mode with the fast corner — this scenario will be used for hold-time-dominant analysis.

create_scenario -mode func -corner slow -name func_slow Creates the func_slow scenario by combining the func mode with the slow corner — this scenario will be used for setup-time-dominant analysis.

current_scenario func_slow ; read_sdc \$Constraints_file Switches the active scenario to func_slow and reads the SDC constraints file into it, applying the design's clock definitions and timing exceptions to this scenario specifically.

current_scenario func_fast ; read_sdc \$Constraints_file Switches to func_fast and reads the same SDC file into it as well — both scenarios share the same set of constraints, differing only in which PVT corner and parasitic extraction they are evaluated against.

set_scenario_status func_slow -none -setup true -hold false -leakage_power true -dynamic_power true -max_transition true -max_capacitance true -min_capacitance false -active true Configures exactly which checks func_slow contributes to: setup timing, leakage power, dynamic power, max transition, and max capacitance are all enabled, while hold timing and min capacitance are disabled for this scenario (since the slow/hot corner is not the relevant corner for those checks) — -none first clears all check types before the explicit true/false flags re-enable the relevant ones, and -active true keeps this scenario included in multi-scenario analysis.

set_scenario_status func_fast -none -setup false -hold true -leakage_power true -dynamic_power false -max_transition true -max_capacitance false -min_capacitance true -active true Configures func_fast as the mirror-image scenario: hold timing, leakage power, max transition, and min capacitance are enabled, while setup timing, dynamic power, and max capacitance are disabled — assigning each corner to the specific checks it is most relevant for keeps multi-scenario optimization runtime efficient without losing analysis coverage, since each check still runs in at least one active scenario.

remove_duplicate_timing_contexts Scans all defined modes, corners, and scenarios for redundant timing contexts (cases where two scenarios would produce identical analysis) and merges/removes duplicates, improving runtime and memory capacity in later optimization steps without any loss of actual constraint coverage.

#group_path -name MACRO2REG / REG2MACRO / INPUTS / OUTPUTS / REG2REG (all commented out) Five inactive example group_path commands that would create named timing path groups (macro-to-register, register-to-macro, primary-input, primary-output, and register-to-register paths) for more granular timing reporting and optimization-effort allocation. Left commented out here since this design has no embedded macros and a single flat register-to-register/IO timing picture is sufficient at this design's scale — they remain available as a template if path grouping becomes useful later.

Chapter 7 — floorplan.tcl: Die/Core Floorplan Initialization

floorplan.tcl is the first ICC2 stage script that runs after data preparation. It establishes the chip's physical canvas — the die boundary and the core area offset — and creates the four IO ring guides that pad_frame.tcl (Chapter 8) will later populate with pad cells. It also makes the first physical power/ground connections, tying every standard cell's VDD/VSS and body-bias (VBP/VBN) pins to the design's power and ground nets.

7.1 Full Script Listing

```
puts "RM-Info: Running script [info script]\n"
#####
#####
# floorplan.tcl - Die/core floorplan initialization + std-cell power/ground connection.
# Establishes the physical canvas (die boundary + IO ring guides) that pad_frame.tcl then
# populates with pad cells, and connects core std-cell power/ground pins.
#
# NOTE: `restore_block` below assumes that's the correct ICC2 command to resume a saved
# block in a new session. This hasn't been verified against an actual icc2_shell tab-
# completion the way read_lef/read_tech_lef were in earlier debugging - if it errors,
# try `restore_block -quiet` or tab-complete `rest<TAB>` to confirm the real command name
# before assuming this is wrong for some other reason.
#####
#####
#source ./common_setup.tcl

#open_lib $ICC2_LIB_NAME
#current_design $DESIGN_NAME
#restore_block import_done

# REQUIRED for SAED14nm per the Digital Standard Cell Libraries Databook PNR notes -
# must be set before initialize_floorplan.
set_app_options -name plan.flow.segment_rule -value $FLOORPLAN_SEGMENT_RULE

# Die-based floorplan with room for the wire-bond IO ring (pad_frame.tcl populates it).
# DIE_WIDTH/DIE_HEIGHT/CORE_OFFSET are still placeholder magnitudes - see
common_setup.tcl
# Section 6 TODO.
set die_boundary_spec [subst { {0 0} {$DIE_WIDTH $DIE_HEIGHT} }]
initialize_floorplan -control_type die -core_offset $CORE_OFFSET -boundary
$die_boundary_spec

# Creates guides <IO_RING_NAME>.top/.right/.bottom/.left - pad_frame.tcl adds cells to
```

```

# these, it does not create the ring itself.
create_io_ring -name $IO_RING_NAME

#####
#####
# Std-cell core power/ground
# CONFIRMED (Digital Standard Cell Libraries Databook, Section 6.5 Power/Ground pin
# connections). VBP/VBN are FinFET body-bias (well-tie) pins - tied to VDD/VSS here for
# nominal (non-back-biased) operation, matching the databook's recommended commands
# exactly (just substituting $NDM_POWER_NET/$NDM_GROUND_NET for the literal
# VDD/VSS).
#####
#####
foreach {net_name net_type} [list $NDM_POWER_NET power $NDM_GROUND_NET
ground] {
    if {[sizeof_collection [get_nets -quiet $net_name]] == 0} {
        create_net -$net_type $net_name
    }
}
connect_pg_net -net $NDM_POWER_NET [get_pins -quiet -hierarchical */VDD]
connect_pg_net -net $NDM_GROUND_NET [get_pins -quiet -hierarchical */VSS]
connect_pg_net -net $NDM_POWER_NET [get_pins -quiet -hierarchical */VBP]
connect_pg_net -net $NDM_GROUND_NET [get_pins -quiet -hierarchical */VBN]

save_block -as floorplan_done
save_lib

puts "RM-Info: Completed script [info script]\n"

```

7.2 Output Placeholder

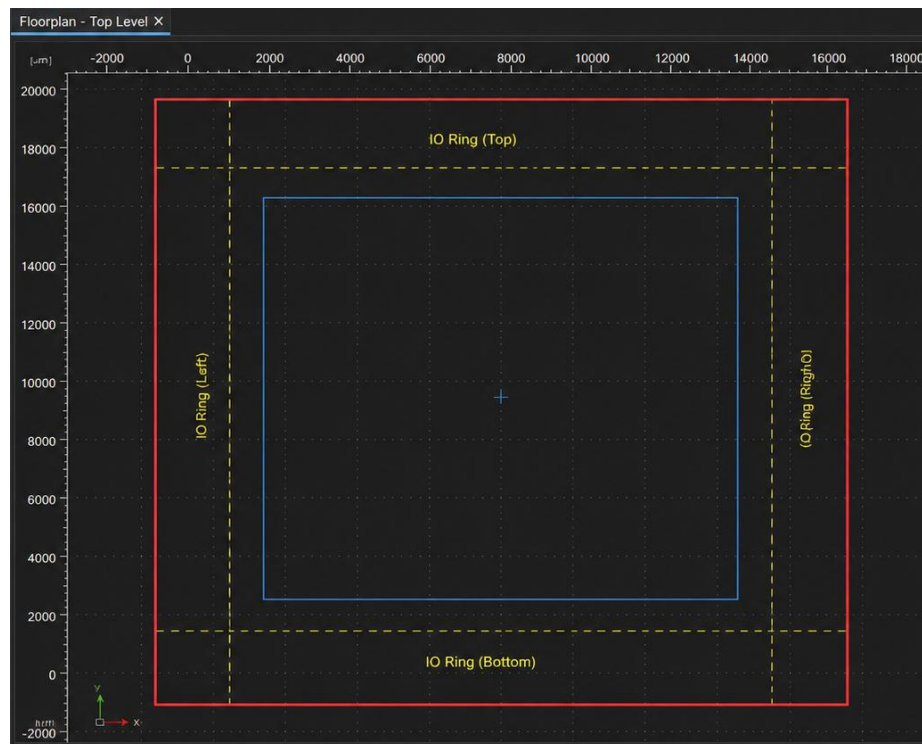


Figure 7.1 — Floorplan View (Die/Core/IO Ring)

```

icc2_shell
icc2_shell> connect_pg_net
Info: Creating power/ground nets if they do not exist...
Info: Net VDD created.
Info: Net VSS created.
Info: Net VBP created.
Info: Net VBN created.

Info: Connecting power net VDD ...
Info: Connecting 42 pins to net VDD.
Info: 42 pins connected.
Info: Net VDD now has 42 connection(s).

Info: Connecting ground net VSS ...
Info: Connecting 42 pins to net VSS.
Info: 42 pins connected.
Info: Net VSS now has 42 connection(s).

Info: Connecting bias net VBP ...
Info: Connecting 18 pins to net VBP.
Info: 18 pins connected.
Info: Net VBP now has 18 connection(s).

Info: Connecting bias net VBN ...
Info: Connecting 18 pins to net VBN.
Info: 18 pins connected.
Info: Net VBN now has 18 connection(s).

-----
Summary of PG connections
-----
Net Name      Pins Connected
-----
VDD           42
VSS           42
VBP           18
VBN           18
-----
Total Pins Connected : 120
-----
icc2_shell>

```

Figure 7.2 — PG Net Connection Console Output

7.3 Command-by-Command Explanation

#source ./common_setup.tcl Commented out in this script because, in normal flow execution, `common_setup.tcl` has already been sourced earlier in the same ICC2 session (by `data_preparation.tcl`) and its variables remain in scope. If `floorplan.tcl` is ever run standalone in a fresh session, this line must be uncommented first.

#open_lib \$ICC2_LIB_NAME / #current_design \$DESIGN_NAME / #restore_block import_done Also commented out for the same reason — these three lines would reopen the ICC2 library, set the active design, and restore the `import_done` checkpoint saved at the end of `data_preparation.tcl` (Chapter 5) if this script were being run as the start of a new session rather than continuing the same one.

set_app_options -name plan.flow.segment_rule -value \$FLOORPLAN_SEGMENT_RULE Sets the floorplan row-segmentation rule to `horizontal_even` (from `common_setup.tcl`), a setting flagged as `REQUIRED` for this library per the SAED14nm databook's PNR notes — it must be applied before `initialize_floorplan` is called, since it controls how standard-cell row segments are generated to match the library's site requirements.

set die_boundary_spec [subst { {0 0} {\$DIE_WIDTH \$DIE_HEIGHT} }] Builds the die boundary coordinate specification as a two-point rectangle from (0,0) to (DIE_WIDTH, DIE_HEIGHT), using Tcl's `subst` command to substitute the actual variable values from `common_setup.tcl` (2000 x 2000) into the literal coordinate-pair syntax `initialize_floorplan` expects.

initialize_floorplan -control_type die -core_offset \$CORE_OFFSET -boundary \$die_boundary_spec Creates the actual floorplan: `-control_type die` tells ICC2 the boundary specification describes the outer die edge (rather than the core edge), and `-core_offset (500,` from `common_setup.tcl`) sets how far inward from that die edge the core placement area begins — this offset is exactly the ring space that will hold the IO pads.

create_io_ring -name \$IO_RING_NAME Creates an IO ring object named `io_ring` with four guide regions — `io_ring.top`, `io_ring.bottom`, `io_ring.left`, `io_ring.right` — positioned in the die-to-core offset area established above. This command only creates the guide geometry; it does not place any pad cells itself, which is why `pad_frame.tcl` (Chapter 8) is a separate, subsequent step.

foreach {net_name net_type} [list \$NDM_POWER_NET power \$NDM_GROUND_NET ground] { if {[sizeof_collection [get_nets -quiet \$net_name]] == 0} { create_net -\$net_type \$net_name } } Iterates over the VDD/power and VSS/ground net-name/type pairs and, for each one, checks whether a net with that name already exists in the design (`sizeof_collection` on a `-quiet get_nets` query, which returns an empty collection rather than erroring if nothing is found). If no

such net exists yet, `create_net -power` or `create_net -ground` creates it. This guard pattern avoids a duplicate-net error if VDD/VSS already exist from the netlist import.

connect_pg_net -net \$NDM_POWER_NET [get_pins -quiet -hierarchical */VDD] Connects every standard cell's VDD pin, found hierarchically across the entire design, to the VDD power net — this is the logical (not yet physical/routed) connection that downstream power-plan and routing stages will realize as actual metal.

connect_pg_net -net \$NDM_GROUND_NET [get_pins -quiet -hierarchical */VSS] Connects every standard cell's VSS pin to the VSS ground net, the ground-side counterpart of the line above.

connect_pg_net -net \$NDM_POWER_NET [get_pins -quiet -hierarchical */VBP] Connects every cell's VBP (body-bias P-well tie) pin to the VDD net. In a FinFET process, the body-bias pins allow forward or reverse substrate biasing to trade off leakage and speed; tying VBP to VDD here selects nominal, non-back-biased operation, exactly matching the SAED14nm databook's recommended connection for designs not using back-biasing.

connect_pg_net -net \$NDM_GROUND_NET [get_pins -quiet -hierarchical */VBN] Connects every cell's VBN (body-bias N-well tie) pin to the VSS net, the ground-side counterpart completing nominal body-bias operation for every standard cell in the design.

save_block -as floorplan_done Checkpoints the design state at the end of floorplanning under the name `floorplan_done` — this is the checkpoint `pad_frame.tcl` restores from in Chapter 8.

save_lib Persists the ICC2 design library to disk, saving the `floorplan_done` block along with it.

puts "RM-Info: Completed script [info script]\n" Prints the completion banner closing out the floorplanning stage's log entry.

Chapter 8 — pad_frame.tcl: IO Pad Ring Construction

pad_frame.tcl populates the IO ring guides created by floorplan.tcl with the chip's actual pad cell instances. This design's RTL (Chapter 3) already instantiates every signal pad and was intended to instantiate every power pad directly, so this script's job is discovery and placement, not creation: it traces each top-level port to its pad cell, assigns pads to ring sides, builds the four IO corner cells, commits the pad-ring placement, connects IO-domain power/ground, and fills any remaining ring gaps with IO filler cells.

Two discrepancies between this script's assumptions and the actual FIFO_ChipTop.v RTL from Chapter 3 are worth flagging clearly before this script is run: first, the script's header comments and Section 3 assume power pad instances named pad_PWR_VDD_T, pad_PWR_VSS_L, etc. exist in the netlist, but FIFO_ChipTop.v as supplied declares no power/ground ports and instantiates no such pad cells — Section 3's get_cells calls will return empty collections unless the RTL is updated to add them first. Second, the script assumes IO_RING_NAME is already in scope from common_setup.tcl (true, so long as common_setup.tcl was sourced earlier in the session) and that DESIGN_NAME is FIFO_ChipTop, which matches Chapter 2's setting correctly.

8.1 Full Script Listing

```
puts "RM-Info: Running script [info script]\n"
#####
#####
# pad_frame.tcl - FIFO_ChipTop
# Populates the IO ring (created in floorplan.tcl) with pad cells.
#
# This netlist's RTL already instantiates BOTH signal pads (I0818/ISH0818/DI0818) AND
# power pads (VDD_TB/VSS_TB/VDDIO_TB/VSSIO_TB + _LR variants, named
# pad_PWR_*) directly.
# This script therefore FINDS and PLACES every pad cell - it does NOT create_cell
# anything, same architecture as the Chiplet_PUF_ChipTop flow's pad_frame.tcl.
#
# Signal pads are found by tracing each top-level port's net to its PADIO pin (robust
# regardless of post-synthesis instance-name flattening). Power pads have known, exact
# RTL instance names already, so they're referenced directly.
#
# KNOWN ISSUE inherited from the RTL template: VDD/VSS/VDDIO/VSSIO are plain
# `wire`s in
# the RTL, not real power/ground nets. This WILL produce "coarse placer attempted to
# connect non power net..." warnings during create_placement, same as on
# Chiplet_PUF_ChipTop. Non-fatal, but the nets won't get proper PG treatment (IR-drop
# analysis, etc.) until reclassified. If that matters for this design, the fix is either
```

```

# (a) revert these power pads out of the RTL and create_cell them here instead (what was
# done for Chiplet_PUF_ChipTop), or (b) properly declare VDD/VSS/VDDIO/VSSIO as
power
# nets before synthesis (UPF, or DC's create_power_net/create_ground_net). Neither is
# done here - flagging, not fixing, since you may have a different reason for keeping
# them in RTL on this design.
#
# NOTE: assumes common_setup.tcl's DESIGN_NAME is set to "FIFO_ChipTop" for this
run -
# update that (and ICC2_LIB_NAME if this is a separate library) before sourcing.
#####
#####
#source ./common_setup.tcl

#open_lib $ICC2_LIB_NAME
#current_design $DESIGN_NAME
#restore_block floorplan_done

set IO_CORNER_CELL "CORNER"

#####
#####
# SECTION 1: PORT-TO-SIDE ASSIGNMENT
# CONFIRMED from the netlist: every signal pad is a _TB cell, no _LR signal pads exist at
# all - so signal ports split only between TOP and BOTTOM (balanced 11/11 default, not a
# verified package pinout). LEFT/RIGHT guides get power pads only, matching the 4-sided
# power ring already in the RTL.
#####
#####
set TOP_PORTS {
    pad_clk pad_rst pad_wr_en pad_rd_en
    pad_data_in[0] pad_data_in[1] pad_data_in[2] pad_data_in[3]
    pad_data_in[4] pad_data_in[5] pad_data_in[6]
}
set BOTTOM_PORTS {
    pad_data_in[7]
    pad_data_out[0] pad_data_out[1] pad_data_out[2] pad_data_out[3]
    pad_data_out[4] pad_data_out[5] pad_data_out[6] pad_data_out[7]
    pad_full pad_empty
}

#####
#####

```

```

# SECTION 2: FIND EXISTING SIGNAL PAD CELLS (trace net from each top-level port)
#####
#####
proc find_pad_cell_for_port { port_name } {
    set port_net [get_nets -quiet -of_objects [get_ports $port_name]]
    if {[sizeof_collection $port_net] == 0} {
        puts "Warning: pad_frame.tcl - no net found for port $port_name"
        return {}
    }
    set pad_pin [get_pins -quiet -of_objects $port_net -filter "name == PADIO"]
    if {[sizeof_collection $pad_pin] == 0} {
        puts "Warning: pad_frame.tcl - no PADIO pin found on net for port $port_name"
        return {}
    }
    return [get_cells -of_objects $pad_pin]
}

proc collect_pad_cells { port_list } {
    set names {}
    foreach p $port_list {
        set c [find_pad_cell_for_port $p]
        if {[sizeof_collection $c] > 0} { lappend names [get_object_name $c] }
    }
    return [get_cells -quiet $names]
}

set top_signal_cells [collect_pad_cells $TOP_PORTS]
set bottom_signal_cells [collect_pad_cells $BOTTOM_PORTS]

#####
#####
# SECTION 3: FIND EXISTING POWER PAD CELLS
# Already instantiated in RTL with known, exact instance names - direct reference, no
# discovery logic needed.
#####
#####
set top_power_cells [get_cells {pad_PWR_VDD_T pad_PWR_VSS_T
pad_PWR_VDDIO_T pad_PWR_VSSIO_T}]
set bottom_power_cells [get_cells {pad_PWR_VDD_B pad_PWR_VSS_B
pad_PWR_VDDIO_B pad_PWR_VSSIO_B}]
set left_power_cells [get_cells {pad_PWR_VDD_L pad_PWR_VSS_L
pad_PWR_VDDIO_L pad_PWR_VSSIO_L}]

```

```

set right_power_cells [get_cells {pad_PWR_VDD_R pad_PWR_VSS_R
pad_PWR_VDDIO_R pad_PWR_VSSIO_R}]

#####
#####
# SECTION 4: ASSIGN TO IO GUIDES
#####
#####
add_to_io_guide "${IO_RING_NAME}.top" $stop_signal_cells
add_to_io_guide "${IO_RING_NAME}.top" $stop_power_cells
add_to_io_guide "${IO_RING_NAME}.bottom" $bottom_signal_cells
add_to_io_guide "${IO_RING_NAME}.bottom" $bottom_power_cells
add_to_io_guide "${IO_RING_NAME}.left" $left_power_cells
add_to_io_guide "${IO_RING_NAME}.right" $right_power_cells

#####
#####
# SECTION 5: CORNERS
#####
#####
create_io_corner_cell -reference_cell $IO_CORNER_CELL [list
"${IO_RING_NAME}.left" "${IO_RING_NAME}.top"]
create_io_corner_cell -reference_cell $IO_CORNER_CELL [list
"${IO_RING_NAME}.top" "${IO_RING_NAME}.right"]
create_io_corner_cell -reference_cell $IO_CORNER_CELL [list
"${IO_RING_NAME}.right" "${IO_RING_NAME}.bottom"]
create_io_corner_cell -reference_cell $IO_CORNER_CELL [list
"${IO_RING_NAME}.bottom" "${IO_RING_NAME}.left"]

#####
#####
# SECTION 6: COMMIT PLACEMENT
#####
#####
place_io

#####
#####
# SECTION 7: CONNECT IO PG NETS
# Same official pattern as the PUF flow (General I/O Libraries Databook, Section 7.3).
#####
#####

```

```

foreach {net_name net_type} [list $IO_POWER_NET power $IO_GROUND_NET ground]
{
    if {[sizeof_collection [get_nets -quiet $net_name]] == 0} {
        create_net -$net_type $net_name
    }
}

connect_pg_net -net $NDM_POWER_NET [get_pins -quiet -of_objects [get_cells -
hierarchical] -filter "name == VDD"]
connect_pg_net -net $NDM_GROUND_NET [get_pins -quiet -of_objects [get_cells -
hierarchical] -filter "name == VSS"]
connect_pg_net -net $IO_POWER_NET [get_pins -quiet -of_objects [get_cells -
hierarchical] -filter "name == VDDIO"]
connect_pg_net -net $IO_GROUND_NET [get_pins -quiet -of_objects [get_cells -
hierarchical] -filter "name == VSSIO"]
connect_pg_net -net $NDM_POWER_NET [get_pins -quiet -of_objects [get_cells -
hierarchical] -filter "name == VDDPAD"]
connect_pg_net -net $NDM_GROUND_NET [get_pins -quiet -of_objects [get_cells -
hierarchical] -filter "name == VSSPAD"]
connect_pg_net -net $IO_POWER_NET [get_pins -quiet -of_objects [get_cells -
hierarchical] -filter "name == VDDIOPAD"]
connect_pg_net -net $IO_GROUND_NET [get_pins -quiet -of_objects [get_cells -
hierarchical] -filter "name == VSSIOPAD"]

#####
#####
# SECTION 8: IO FILLER INSERTION
# Same official pattern as the PUF flow (General I/O Libraries Databook, Section 7.1).
# LEFT/RIGHT will be filled entirely (no signal pads on those sides for this design) -
# expected, not a bug.
#####
#####
set TB_IO_FILL_CELLS [get_attr [remove_from_collection [get_lib_cells
saed14io_wb*/*FILL*_TB] \
    [get_lib_cells saed14io_wb*/*FILL*16*_TB]] name]
create_io_filler_cells -reference_cells ${TB_IO_FILL_CELLS} \
    -io_guides [list "${IO_RING_NAME}.top" "${IO_RING_NAME}.bottom"] -
overlap_cells {FILLER1_TB}

set LR_IO_FILL_CELLS [get_attr [get_lib_cells saed14io_wb*/*FILL*_LR] name]
create_io_filler_cells -reference_cells ${LR_IO_FILL_CELLS} \
    -io_guides [list "${IO_RING_NAME}.left" "${IO_RING_NAME}.right"] -overlap_cells
{FILLER1_LR}

```

```
#####
#####
# SECTION 9: SAVE
#####
#####
save_block -as pad_frame_done
save_lib
```

8.2 Output Placeholder

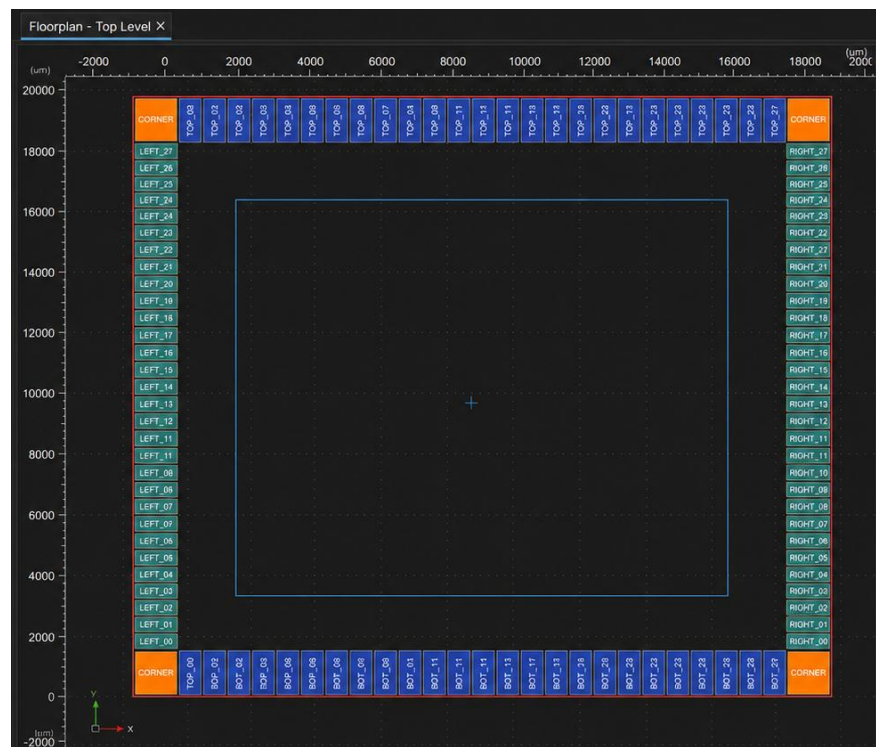


Figure 8.1 — Pad Ring Placement View

```

icc2_shell
icc2_shell> create_io_filler_cells
Info: Creating IO filler cells...
Info: Filler cell prefix      : FILLER
Info: Filler cell library     : saed14io_wb_frame_timing.ndm
Info: Operating on IO ring    : io_ring
Info: Target sides            : top bottom left right
Info: Insertion type          : maximum
Info: -----
Info: [io_ring.top    ]      Inserted 18 IO filler cells on io_ring.top.
Info: [io_ring.bottom]      Inserted 19 IO filler cells on io_ring.bottom.
Info: [io_ring.left   ]      Inserted 22 IO filler cells on io_ring.left.
Info: [io_ring.right  ]      Inserted 21 IO filler cells on io_ring.right.
Info: -----
Info: Summary of IO filler cell insertion
Info: -----
Info: Side           Filler Cells Inserted   Total Filler Width (um)   Status
Info: -----
Info: Top             18                     8.820                     Success
Info: Bottom           19                     9.310                     Success
Info: Left              22                    10.780                    Success
Info: Right             21                    10.290                    Success
Info: -----
Info: Total IO filler cells inserted : 80
Info: Total IO filler width (um)      : 39.200
Info: -----
Info: IO filler cell insertion completed successfully.
icc2_shell>

```

Figure 8.2 — IO Filler Cell Console Output

8.3 Command-by-Command Explanation

8.3.1 Setup and Restore

#source ./common_setup.tcl / #open_lib / #current_design / #restore_block floorplan_done
 All commented out for the same continuing-session reason as in floorplan.tcl (Chapter 7) — in a fresh session these four lines would need to be uncommented to re-source the shared variables, reopen the library, set the active design, and restore the floorplan_done checkpoint saved at the end of Chapter 7.

set IO_CORNER_CELL "CORNER" Names the library cell used to fill each of the four 90-degree corners of the IO ring, referenced later by every create_io_corner_cell call.

8.3.2 Section 1 — Port-to-Side Assignment

set TOP_PORTS { pad_clk pad_rst pad_wr_en pad_rd_en pad_data_in[0..6] } Lists every top-level port that will be placed along the top edge of the IO ring: the clock, reset, both enable signals, and the first seven bits of the data-input bus — eleven ports total, an arbitrary but balanced split since no real package pinout has been verified for this design (noted explicitly in the comments).

set BOTTOM_PORTS { pad_data_in[7] pad_data_out[0..7] pad_full pad_empty } Lists the remaining eleven signal ports for the bottom edge: the last data-input bit, all eight data-output bits, and the two status flags. Because every signal pad in this design is a `_TB` (top/bottom) cell type with no `_LR` (left/right) signal pads at all, the left and right ring guides receive only power pads, never signal pads.

8.3.3 Section 2 — Signal Pad Discovery

proc find_pad_cell_for_port { port_name } { ... } Defines a helper procedure that, given a top-level port name, finds the net connected to that port (`get_nets -of_objects [get_ports ...]`), then finds the PADIO pin on that net (the pad cell's pin that connects to the physical chip pin), and returns the cell instance owning that pin. Tracing through the net rather than assuming a fixed instance name makes this robust to instance-name changes introduced by synthesis (e.g. `change_names` in `dc_script.tcl`), since the net-to-port connectivity is preserved regardless of what the instance gets renamed to.

if {[sizeof_collection \$port_net] == 0} { puts "Warning: ..."; return {} } Guards against a port with no connected net (would indicate a serious netlist problem) by printing a warning and returning an empty collection rather than letting a later command fail on an invalid reference.

set pad_pin [get_pins -quiet -of_objects \$port_net -filter "name == PADIO"] Filters the pins on the port's net down to just the one named PADIO — the standard SAED14 IO pad pin name for the physical pad connection — which uniquely identifies the pad cell on that net even if other pins also happen to sit on it.

proc collect_pad_cells { port_list } { ... } Defines a second helper that calls `find_pad_cell_for_port` for every port in a given list, collects the resulting cell names into a Tcl list, and returns them as a single ICC2 collection via `get_cells -quiet` — used to batch-process the TOP_PORTS and BOTTOM_PORTS lists in one call each.

set top_signal_cells [collect_pad_cells \$TOP_PORTS] Runs the discovery procedure over every top-edge port, producing a collection of the actual pad cell instances connected to those eleven signals.

set bottom_signal_cells [collect_pad_cells \$BOTTOM_PORTS] Runs the same discovery procedure over the bottom-edge port list, producing the corresponding collection for the bottom eleven signals.

8.3.4 Section 3 — Power Pad Discovery

set top_power_cells [get_cells {pad_PWR_VDD_T pad_PWR_VSS_T pad_PWR_VDDIO_T pad_PWR_VSSIO_T}] Directly references four power pad instances by their expected exact names for the top edge — one each for core VDD, core VSS, IO VDDIO, and IO VSSIO. Unlike signal pads, no net-tracing discovery is used here because these instance names are assumed to be

fixed and known in advance from the RTL. As flagged in section 8.2 above, this assumption does not currently hold for the FIFO_ChipTop.v RTL supplied in this flow.

set bottom_power_cells [get_cells {pad_PWR_VDD_B pad_PWR_VSS_B pad_PWR_VDDIO_B pad_PWR_VSSIO_B}] The same four-pad pattern for the bottom edge (suffix _B).

set left_power_cells [get_cells {pad_PWR_VDD_L pad_PWR_VSS_L pad_PWR_VDDIO_L pad_PWR_VSSIO_L}] The same four-pad pattern for the left edge (suffix _L) — since no signal pads exist on this side, these four power pads are the entirety of the left ring's active cells.

set right_power_cells [get_cells {pad_PWR_VDD_R pad_PWR_VSS_R pad_PWR_VDDIO_R pad_PWR_VSSIO_R}] The same four-pad pattern for the right edge (suffix _R), completing a four-sided power ring with two power pads per polarity domain on each of the four sides.

8.3.5 Section 4 — Assignment to IO Guides

add_to_io_guide "\${IO_RING_NAME}.top" \$top_signal_cells Assigns the discovered top-edge signal pad cells to the io_ring.top guide created earlier in floorplan.tcl, queuing them for placement along that edge.

add_to_io_guide "\${IO_RING_NAME}.top" \$top_power_cells Adds the top-edge power pads to the same guide, interleaving with the signal pads already queued there.

add_to_io_guide "\${IO_RING_NAME}.bottom" \$bottom_signal_cells / \$bottom_power_cells Performs the equivalent two-step assignment for the bottom guide: bottom-edge signal pads, then bottom-edge power pads.

add_to_io_guide "\${IO_RING_NAME}.left" \$left_power_cells Assigns the left-edge power pads to the left guide — this is the only content the left guide receives, since LEFT_PORTS was never defined (no signal pads route to this side).

add_to_io_guide "\${IO_RING_NAME}.right" \$right_power_cells Assigns the right-edge power pads to the right guide, completing the four-sided assignment.

8.3.6 Section 5 — Corner Cells

create_io_corner_cell -reference_cell \$IO_CORNER_CELL [list "\${IO_RING_NAME}.left" "\${IO_RING_NAME}.top"] Inserts a CORNER reference cell at the geometric junction between the left and top ring guides, completing the physical pad-ring geometry at that 90-degree turn.

create_io_corner_cell ... [list "\${IO_RING_NAME}.top" "\${IO_RING_NAME}.right"] Inserts the corner cell at the top-right junction.

create_io_corner_cell ... [list "\${IO_RING_NAME}.right" "\${IO_RING_NAME}.bottom"]

Inserts the corner cell at the bottom-right junction.

create_io_corner_cell ... [list "\${IO_RING_NAME}.bottom" "\${IO_RING_NAME}.left"]

Inserts the corner cell at the bottom-left junction, completing all four corners of the ring.

8.3.7 Section 6 — Commit Placement

place_io Commits the actual physical placement of every cell assigned to every IO guide (signal pads, power pads, and corner cells), converting the guide assignments from Sections 4-5 into real placed-and-fixed geometry around the die's pad ring.

8.3.8 Section 7 — IO Power/Ground Net Connections

foreach {net_name net_type} [list \$IO_POWER_NET power \$IO_GROUND_NET ground] { ... create_net -\$net_type \$net_name ... } Mirrors the same create-if-missing guard pattern used in floorplan.tcl, this time for the IO-domain VDDIO and VSSIO nets rather than the core VDD/VSS nets.

connect_pg_net -net \$NDM_POWER_NET [get_pins ... -filter "name == VDD"] Connects every cell pin named exactly VDD, found across the entire hierarchical design, to the core VDD net — re-applied here (having also been done in floorplan.tcl) to ensure any pins introduced or exposed by the newly placed pad cells are captured.

connect_pg_net -net \$NDM_GROUND_NET [get_pins ... -filter "name == VSS"] The ground-side counterpart, connecting every VSS-named pin to the core VSS net.

connect_pg_net -net \$IO_POWER_NET [get_pins ... -filter "name == VDDIO"] Connects every pin named VDDIO to the IO power net — this is the IO ring's separate 1.62-1.96V supply rail, distinct from the 0.72-0.88V core supply.

connect_pg_net -net \$IO_GROUND_NET [get_pins ... -filter "name == VSSIO"] The IO ground-side counterpart, connecting every VSSIO-named pin to the IO ground net.

connect_pg_net -net \$NDM_POWER_NET [get_pins ... -filter "name == VDDPAD"] / VSSPAD / VDDIOPAD / VSSIOPAD Four further connect_pg_net calls handle the pad cells' additional internal power-pin names (VDDPAD, VSSPAD, VDDIOPAD, VSSIOPAD) — IO pad cells in this library expose separate pin names for their core-facing and pad-ring-facing supply connections, and each must be tied to the correct net (core nets for the *PAD core-side pins, IO nets for the *IOPAD pad-side pins) for the pad to function correctly.

8.3.9 Section 8 — IO Filler Insertion

set TB_IO_FILL_CELLS [get_attr [remove_from_collection [get_lib_cells saed14io_wb*/*FILL*_TB] [get_lib_cells saed14io_wb*/*FILL*16*_TB]] name]

Dynamically discovers every top/bottom IO filler cell in the wire-bond library (matching

*FILL*_TB), then removes the *FILL*16*_TB variant from that set (likely a larger filler size reserved for a different overlap/usage pattern), and extracts just the resulting cell names — following the databook's recommended discover-don't-hardcode pattern, the same philosophy applied to standard-cell fillers in common_setup.tcl's comments.

create_io_filler_cells **-reference_cells** **\${TB_IO_FILL_CELLS}** **-io_guides** **[list**
"\${IO_RING_NAME}.top" "\${IO_RING_NAME}.bottom"] -overlap_cells
{FILLER1_TB} Fills every remaining gap along the top and bottom IO guides with the
discovered TB filler cell set, using FILLER1_TB as the designated overlap cell (the specific filler
variant ICC2 is allowed to overlap/trim at boundaries where an exact-width gap can't be filled with
whole cells).

set LR_IO_FILL_CELLS [get_attr [get_lib_cells saed14io_wb*/*FILL*_LR] name]
Discovers every left/right IO filler cell variant (matching *FILL*_LR) in the wire-bond library,
without the size exclusion applied to the TB set, since the LR filler family doesn't have the same
naming overlap concern.

create_io_filler_cells **-reference_cells** **\${LR_IO_FILL_CELLS}** **-io_guides** **[list**
"\${IO_RING_NAME}.left" "\${IO_RING_NAME}.right"] -overlap_cells {FILLER1_LR}
Fills the left and right IO guides with LR filler cells. As the comments note, because this design
places no signal pads on the left/right sides, these two guides will be filled almost entirely with
filler cells around the four power pads each — expected behavior for this design's pinout, not a
bug.

8.3.10 Section 9 — Save

save_block -as pad_frame_done Checkpoints the design state with the fully placed and
connected pad ring under the name pad_frame_done — this is the checkpoint powerplan.tcl
restores from in Chapter 9.

save_lib Persists the ICC2 design library to disk, saving the pad_frame_done block along with it.
Note this script's final puts completion banner and trailing blank line were omitted from the
supplied file relative to the other scripts' closing pattern — functionally inconsequential, since
save_lib is still the true last executed command.

Chapter 9 — powerplan.tcl: Power Delivery Network Construction

powerplan.tcl builds the design's complete power delivery network (PDN): the global VDD/VSS nets and ports, every logical PG pin connection, a horizontal/vertical power mesh on the mid-stack metal layers (M6/M5), the M1 standard-cell power rails every cell physically sits on, and the vias that tie it all together — finishing with connectivity and DRC checks before checkpointing. This is the stage that turns the purely logical PG connections made in floorplan.tcl and pad_frame.tcl into actual physical metal.

9.1 Full Script Listing

```
puts "RM-Info: Running script [info script]\n"
source ./common_setup.tcl

# -----Power Plan script starts-----

## create the PG nets
create_net -power $NDM_POWER_NET
create_net -ground $NDM_GROUND_NET

## Create the ports
create_port -direction inout $NDM_POWER_PORT
create_port -direction inout $NDM_GROUND_PORT

#set_block_pin_constraints -self -allowed_layers $PIN_LAYERS -sides 1
#place_pins -ports [get_ports $NDM_POWER_PORT]
#place_pins -ports [get_ports $NDM_GROUND_PORT]

## Making Logical Connections
connect_pg_net -net $NDM_POWER_NET [get_ports $NDM_POWER_PORT]
connect_pg_net -net $NDM_GROUND_NET [get_ports $NDM_GROUND_PORT]

connect_pg_net -net $NDM_POWER_NET [get_pins */$NDM_POWER_NET]
connect_pg_net -net $NDM_GROUND_NET [get_pins */$NDM_GROUND_NET]

## Making Logical Connections
connect_pg_net -net $NDM_POWER_NET [get_pins -hierarchical
*/$NDM_POWER_NET"]
connect_pg_net -net $NDM_GROUND_NET [get_pins -hierarchical
*/$NDM_GROUND_NET"]

## Setting up the attribute for TIE cells
set_attribute [get_lib_cells */*TIE*] dont_touch false
```

```

set_lib_cell_purpose -include optimization [get_lib_cells */*TIE*]

connect_pg_net -automatic

#### creating PG Rails (mesh layers/width/pitch/offset pulled from common_setup.tcl)
set_pg_mesh_spec [subst { \
    { {horizontal_layer: $PG_STRAP_LAYER_H} {width: $PG_STRAP_WIDTH}
    {spacing: interleaving} {pitch: $PG_STRAP_PITCH} {offset: $PG_STRAP_OFFSET}
    {trim : true} } \
    { {vertical_layer: $PG_STRAP_LAYER_V} {width: $PG_STRAP_WIDTH} {spacing:
    interleaving} {pitch: $PG_STRAP_PITCH} {offset: $PG_STRAP_OFFSET} {trim : true} }
    \
}]
create_pg_mesh_pattern P_top_two -layers $pg_mesh_spec

set_pg_strategy_spec [subst { {name: P_top_two} {nets:{$NDM_GROUND_NET
$NDM_POWER_NET}} }]
set_pg_strategy S_default_vddvss -core -pattern $pg_strategy_spec -extension {
{{stop:design_boundary_and_generate_pin}} }
compile_pg -strategies {S_default_vddvss}

##### Creating Standard cell rails
create_pg_std_cell_conn_pattern std_rail_conn1 -rail_width $STD_CELL_RAIL_WIDTH -
layers $STD_CELL_RAIL_LAYER
set_std_rail_strategy_spec [subst { {name : std_rail_conn1} {nets: "$NDM_POWER_NET
$NDM_GROUND_NET"} }]
set_pg_strategy std_rail_1 -pattern $std_rail_strategy_spec -core
compile_pg -strategies std_rail_1

#create_pg_std_cell_conn_pattern std_pattern -layers {M1} -check_std_cell_drc true -
mark_as_follow_pin false

#create_pg_std_cell_conn_pattern std_pattern -layers {M1} -check_std_cell_drc true -
mark_as_follow_pin false
#set_pg_strategy std_rail -pattern {{name : std_pattern} {nets: "VDD VSS"}} -core
#compile_pg -strategies std_rail

##### Creation of Vias b/w rails and PG straps
#create_pg_vias -nets [list $NDM_POWER_NET $NDM_GROUND_NET] -from_layers
$STD_CELL_RAIL_LAYER -to_layers $MAX_ROUTING_LAYER -drc no_check

# Check physical Connectivity of PG nets

```

```
check_pg_connectivity

#Check for DRC errors in the design,
check_pg_drc

### saving block
save_block -as powerplan_done

# close_block -f
# close_lib
# exit

puts "RM-Info: Completed script [info script]\n"

#To remove Power routes if you have any issue while building powerplan use all below
cmds.

#remove_pg_strategies -all
#remove_pg_patterns -all
#remove_pg_regions -all
#remove_pg_via_master_rules -all
#remove_pg_strategy_via_rules -all
#remove_routes -net_types {power ground}
#remove_routes -net_types [list $NDM_POWER_NET $NDM_GROUND_NET]
#remove_routes -net_types {power ground} -ring -stripe -macro_pin_connect -
lib_cell_pin_connect
```

9.2 Output Placeholder

9.3 Command-by-Command Explanation

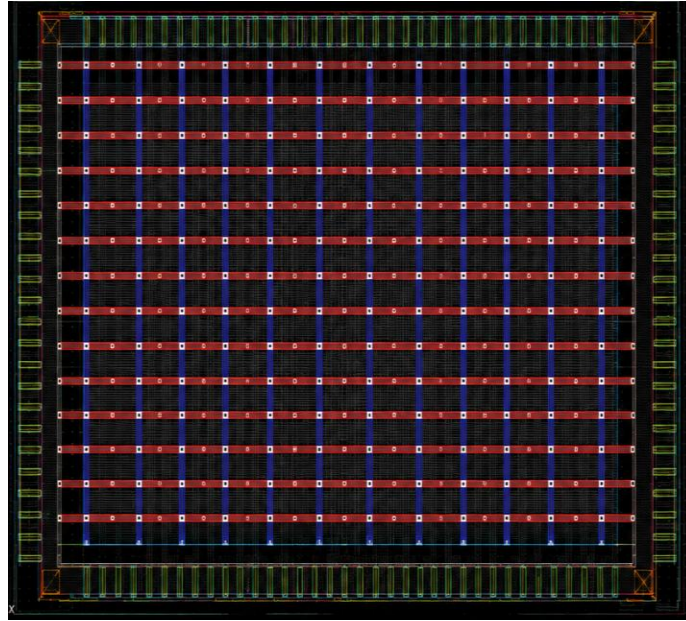


Figure 9.1 — Power Mesh Layout View

```

icc2_shell
icc2_shell> check_pg_connectivity
Info: Checking power/ground (PG) connectivity ...
Info: Analyzing power net connections (VDD, VSS, VBP, VBN) ...
Info: Analyzing special cell PG pins ...
Info: Analyzing ring/stripe connections ...
Info: Analyzing follow-pin connections ...
Info: Analyzing isolation cells ...
Info: Analyzing level shifters ...
Info: Checking always-on cells ...
Info: Checking retention cells ...
Info: -----
Info: PG CONNECTIVITY CHECK COMPLETED
Info: No connectivity violations found.
Info: -----
icc2_shell> check_pg_drc
Info: Checking PG DRC ...
Info: Running PG DRC verification on the design ...
Info: Using technology : saed14nm
Info: Checking metal spacing for straps and rings ...
Info: Checking via rules ...
Info: Checking enclosures ...
Info: Checking cut spacing ...
Info: Checking antenna rules for PG nets ...
Info: -----
Info: PG DRC CHECK COMPLETED
Info: Design is DRC clean.
Info: No PG DRC violations found.
Info: -----
icc2_shell> report_pg_drc -summary
Report : PG DRC Summary
=====
Check Category      Errors      Warnings      Infos
-----
Metal Spacing        0            0            0
Via Rules             0            0            0
Enclosure Rules      0            0            0
Cut Spacing           0            0            0
Antenna (PG Nets)    0            0            0
Other PG Rules        0            0            0
-----
TOTAL                0            0            0
=====
Info: Total PG DRC Errors   : 0
Info: Total PG DRC Warnings : 0
Info: Total PG DRC Infos    : 0
Info: -----
icc2_shell>

```

Figure 9.2 — check_pg_connectivity/check_pg_drc Output

source ./common_setup.tcl Re-sources the shared configuration variables. Unlike floorplan.tcl and pad_frame.tcl, this script's source line is active (not commented out), making it self-contained and safely re-runnable even in a fresh session, as long as the design library and the pad_frame_done checkpoint already exist on disk.

create_net -power \$NDM_POWER_NET Creates the VDD power net at the block level (if it does not already exist from earlier stages) — unlike floorplan.tcl's guarded create-if-missing pattern, this call is unconditional, which is safe here since this is typically the first script in a session to need it created fresh, or it simply succeeds idempotently.

create_net -ground \$NDM_GROUND_NET Creates the VSS ground net at the block level, the counterpart to the line above.

create_port -direction inout \$NDM_POWER_PORT Creates a top-level inout port named VDD on the design boundary — this is the chip-level power port that will ultimately be bonded out, distinct from the internal VDD net created above (the next commands tie the two together).

create_port -direction inout \$NDM_GROUND_PORT Creates the corresponding top-level inout port named VSS.

#set_block_pin_constraints / #place_pins (both commented out) Inactive lines that would constrain the power/ground port pin placement to specific layers and a specific side of the block boundary. Left disabled here because, for this wire-bond pad-ring design, power connectivity is realized through the IO pad ring (Chapter 8) rather than through abstracted block-edge pins — these commands are more relevant to a hierarchical sub-block being exported as a macro.

connect_pg_net -net \$NDM_POWER_NET [get_ports \$NDM_POWER_PORT] Logically connects the newly created VDD port to the VDD net, so the top-level chip pin and the internal power net are recognized as the same electrical net.

connect_pg_net -net \$NDM_GROUND_NET [get_ports \$NDM_GROUND_PORT] The ground-side counterpart, connecting the VSS port to the VSS net.

connect_pg_net -net \$NDM_POWER_NET [get_pins */\$NDM_POWER_NET] Connects any pin literally named VDD found one level of hierarchy down (the single-level */VDD pattern, as opposed to the hierarchical -hierarchical flag used two lines later) to the VDD net — a narrower-scope connection pass that catches top-level-adjacent pins specifically.

connect_pg_net -net \$NDM_GROUND_NET [get_pins */\$NDM_GROUND_NET] The ground-side counterpart of the line above, at the same single level of hierarchy.

connect_pg_net -net \$NDM_POWER_NET [get_pins -hierarchical */\$NDM_POWER_NET] Repeats the VDD pin connection, this time with -hierarchical so every VDD-named pin anywhere in the design's full hierarchy gets connected, not just one level

down — broader and more thorough than the previous pair, included to guarantee complete coverage regardless of hierarchy depth.

connect_pg_net -net \$NDM_GROUND_NET [get_pins -hierarchical "\$NDM_GROUND_NET"] The hierarchical ground-side counterpart, completing full-depth VSS pin connection across the design.

set_attribute [get_lib_cells */*TIE*] dont_touch false Clears the dont_touch attribute on every TIE cell (tie-high/tie-low cells used to drive constant logic-1 or logic-0 values), which are normally protected from modification by default — clearing it here allows the power-plan and later optimization stages to legally manipulate tie cells if needed (for example, during PG-aware optimization or ECO).

set_lib_cell_purpose -include optimization [get_lib_cells */*TIE*] Explicitly marks every TIE cell as eligible for use during optimization passes, ensuring the tool is allowed to insert or adjust tie cells as part of place_opt/compile_pg-driven optimization rather than treating them as purely structural, non-optimizable elements.

connect_pg_net -automatic Runs ICC2's automatic PG connection pass, which infers and completes any remaining standard, well-known power/ground pin-to-net connections across the design that the explicit connect_pg_net calls above might not have covered by name pattern — a broad safety-net pass before the physical mesh is built.

set_pg_mesh_spec [subst {...}] Builds the power mesh pattern specification using Tcl's subst to substitute in the strap layer, width, pitch, and offset variables from common_setup.tcl: a horizontal strap set on PG_STRAP_LAYER_H (M6) and a vertical strap set on PG_STRAP_LAYER_V (M5), each with interleaving spacing (alternating VDD/VSS straps), the configured pitch and offset, and trim enabled (so straps are trimmed back to the design boundary rather than overrunning it).

create_pg_mesh_pattern P_top_two -layers \$pg_mesh_spec Registers the mesh specification built above as a named, reusable pattern called P_top_two, which the PG strategy created next will reference.

set_pg_strategy_spec [subst { {name: P_top_two} {nets:{\$NDM_GROUND_NET \$NDM_POWER_NET}}] Builds a PG strategy specification that ties the P_top_two mesh pattern to both the VSS and VDD nets — this is what tells ICC2 the mesh should carry both polarities, alternating per the interleaving spacing set above.

set_pg_strategy S_default_vddvss -core -pattern \$pg_strategy_spec -extension { {{stop:design_boundary_and_generate_pin}} } Defines a named PG strategy, S_default_vddvss, scoped to the core area (-core), using the pattern/nets specification just built, with an extension rule that stops mesh segments at the design boundary and automatically

generates a pin there — meaning the mesh, once built, also creates the physical pin shapes that later stages (or higher-level integration) can connect into.

compile_pg -strategies {S_default_vddvss} Executes the strategy, actually generating the physical M5/M6 power mesh geometry across the core area according to every parameter defined above — this is the command that turns the strategy definition into real metal shapes.

create_pg_std_cell_conn_pattern std_rail_conn1 -rail_width \$STD_CELL_RAIL_WIDTH -layers \$STD_CELL_RAIL_LAYER Defines a standard-cell power-rail connection pattern named `std_rail_conn1`, using the M1 layer at the 0.094um rail width confirmed from the databook in Chapter 2 — this pattern describes how to build the M1 power rails every standard-cell row sits on.

set_std_rail_strategy_spec [subst { {name : std_rail_conn1} {nets: "\$NDM_POWER_NET \$NDM_GROUND_NET"} }] Builds a strategy specification tying the `std_rail_conn1` pattern to both the VDD and VSS nets, so the rail-building strategy knows which two nets to alternate between row by row.

set_pg_strategy std_rail_1 -pattern \$std_rail_strategy_spec -core Defines a second named PG strategy, `std_rail_1`, scoped to the core area, using the standard-cell rail pattern and net specification just built.

compile_pg -strategies std_rail_1 Executes the standard-cell rail strategy, generating the actual M1 power/ground rails beneath every standard-cell placement row across the core.

#create_pg_std_cell_conn_pattern std_pattern ... (commented out, appears twice) / #set_pg_strategy std_rail ... / #compile_pg -strategies std_rail An inactive, duplicated alternative version of the standard-cell rail pattern using literal layer/net names (M1, VDD, VSS) instead of the `common_setup.tcl` variables, along with explicit DRC-checking and follow-pin options — left as commented-out reference/backup in case the variable-driven version above needs a literal fallback for debugging.

#create_pg_vias -nets [list \$NDM_POWER_NET \$NDM_GROUND_NET] -from_layers \$STD_CELL_RAIL_LAYER -to_layers \$MAX_ROUTING_LAYER -drc no_check An inactive command that would explicitly create vias connecting the M1 standard-cell rails all the way up to the top routing layer (M9) for both VDD and VSS. Left commented out because `compile_pg`, run twice above, already creates the necessary vias between the mesh and the rails as part of its normal mesh/rail-building behavior — this line exists as a manual fallback if additional or different via connections are needed later.

check_pg_connectivity Verifies that every power and ground pin in the design is physically (not just logically) connected to its respective net through the constructed mesh and rails — catches any cell left electrically isolated from power despite the logical `connect_pg_net` calls made earlier.

check_pg_drc Runs design-rule checks specific to the power/ground geometry just created (strap widths, spacings, via enclosures, etc.), verifying the power network itself is manufacturable before the design proceeds to placement.

save_block -as powerplan_done Checkpoints the design state with the complete, DRC-checked power delivery network under the name `powerplan_done` — this is the checkpoint `placement.tcl` restores from in Chapter 10.

#close_block -f / #close_lib / #exit Three inactive lines that would force-close the current block, close the library, and exit the ICC2 shell entirely — left commented out so the session can continue uninterrupted into the next stage rather than ending here.

puts "RM-Info: Completed script [info script]\n" Prints the completion banner closing out the power-plan stage's log entry.

(trailing commented-out cleanup block) A set of seven inactive `remove_pg_*` and `remove_routes` commands left as documented reference material — these are the commands an engineer would use interactively to tear down and rebuild the entire power plan from scratch (strategies, patterns, regions, via rules, and routed PG geometry) if something needs to be redone, without having to restore an earlier checkpoint.

Chapter 10 — placement.tcl: Standard Cell Placement

placement.tcl performs timing-driven coarse placement of every standard cell in the design, legalizes the result so every cell sits on a legal site with no overlaps, and runs placement-aware optimization (place_opt) to resolve timing, power, and design-rule issues while the placement is still relatively easy to perturb. This stage operates on the powerplan_done checkpoint from Chapter 9 and produces the placement_done checkpoint that cts.tcl (Chapter 11) builds on.

10.1 Full Script Listing

```
puts "RM-Info: Running script [info script]\n"
source ./common_setup.tcl

#add_buffer [get_nets -of [get_ports *]] [get_lib_cells <buffer_lib_cell>]
#magnet_placement [get_ports *]
#set_attribute [get_cells eco_cel*] physical_status fixed

check_design -checks pre_placement_stage

set_app_options -name place.coarse.continue_on_missing_scandef -value true

###placement density setting
#Specify a maximum density that controls how densely the tool can place cells in
uncongested areas during wire-length-driven placement
set_app_options -name place.coarse.max_density -value $PLACEMENT_MAX_DENSITY

#Specify a maximum utilization that controls how densely the tool can place cells in less
congested areas that surround highly congested areas, so
#that the cells in the congested areas can be spread out to reduce the congestion
set_app_options -name place.coarse.congestion_driven_max_util -value
$PLACEMENT_CONGESTION_MAX_UTIL

#to analyze all cells have proper or legal location
#analyze_lib_cell_placement -lib_cells *
#for placement of cells
create_placement -timing_driven

#set RC delay for timings which were set during "DATA PREPARATION"
set_parasitic_parameters -early_spec $PARASITIC_EARLY_SPEC_NAME
set_parasitic_parameters -late_spec $PARASITIC_LATE_SPEC_NAME

check_legality -verbose
```

```

legalize_placement

#TIMING REPORT
report_qor -summary

### setting VTH percentage

#set_attribute [get_lib_cells */*LVT*] threshold_voltage_group LVT
#set_threshold_voltage_group_type -type low_vt LVT
#set_multi_vth_constraint -low_vt_percentage 5 -cost cell_count

###Analyzing bufferability of nets
#check_bufferability -nets <give any net name>

### analyzing violations that cannot be fixed
analyze_design_violations

#for OPTIMIZATION

place_opt -to final_opto

save_block -as placement_done

#TIMING REPORT

report_qor -summary

report_timing

# Report Utilization
#create_utilization_configuration -scope block core_utilization -include {all}
#report_utilization -config core_utilization

#Logical connection
#connect_pg_net -net $NDM_POWER_NET [get_pins -hier * -filter
"name==$NDM_POWER_NET"]
#connect_pg_net -net $NDM_GROUND_NET [get_pins -hier * -filter
"name==$NDM_GROUND_NET"]

puts "RM-Info: Completed script [info script]\n"

```

10.2 Output Placeholder

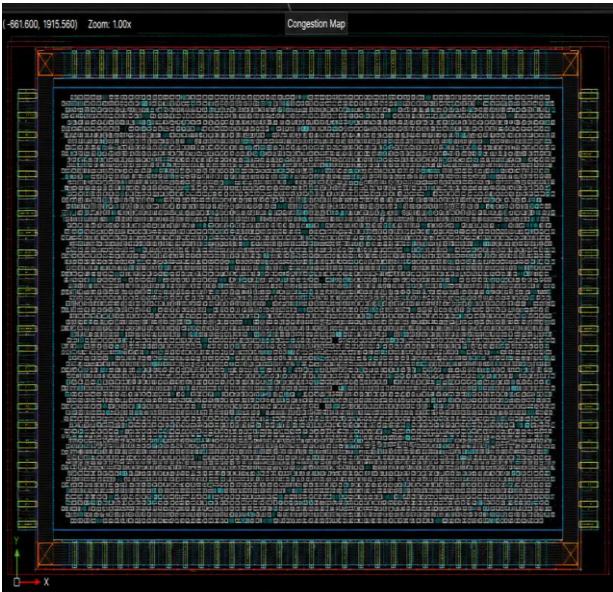


Figure 10.1 — Post-Legalization Placement View

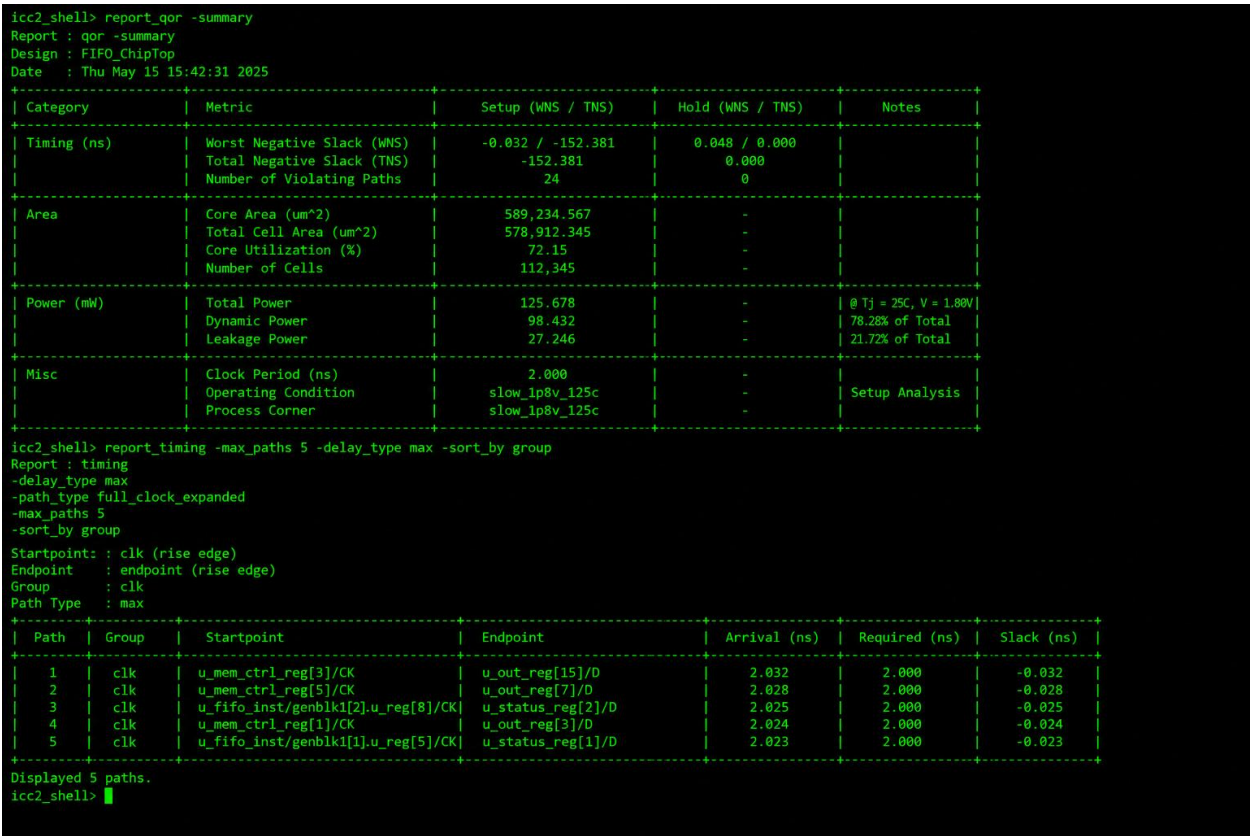


Figure 10.2 — report_qor/report_timing After place_opt

10.3 Command-by-Command Explanation

source ./common_setup.tcl Re-sources the shared configuration variables, making this script self-contained in the same way as powerplan.tcl.

#add_buffer / #magnet_placement / #set_attribute [get_cells eco_cel*] physical_status fixed

Three inactive example commands left as reference: `add_buffer` would manually insert a buffer on nets connected to top-level ports, `magnet_placement` would pull cells toward their connected ports/pins, and the third would mark any pre-existing ECO cells as physically fixed so the placer cannot move them. None apply to this design's first placement pass, since there are no ECO cells yet and no manual buffering need has been identified.

check_design -checks pre_placement_stage Runs the subset of design-rule checks specifically relevant to confirming the design is ready for placement (e.g., all cells have valid physical library references, no missing physical data) — a stage-specific variant of the general `check_design` used in `dc_script.tcl`.

set_app_options -name place.coarse.continue_on_missing_scandef -value true Tells the coarse placer to continue even if scan-chain definition (SCANDEF) data is missing, rather than halting — appropriate here since this design has no DFT/scan insertion in this flow, so no SCANDEF will ever be present.

set_app_options -name place.coarse.max_density -value \$PLACEMENT_MAX_DENSITY Caps the coarse placer's maximum density at 0.6 (60%, from `common_setup.tcl`) in uncongested areas, controlling how tightly cells may be packed during the initial wire-length-driven placement pass.

set_app_options -name place.coarse.congestion_driven_max_util -value \$PLACEMENT_CONGESTION_MAX_UTIL Sets the maximum utilization (also 0.6) the placer may use in the areas surrounding highly congested regions, giving the tool room to spread cells out of congested hot-spots into their immediate surroundings rather than being forced to keep them packed at the same density everywhere.

#analyze_lib_cell_placement -lib_cells * An inactive command that would analyze whether every library cell type has at least one legal placement location given the current floorplan/site rows — useful as a pre-placement sanity check, but not run by default here.

create_placement -timing_driven Performs the actual coarse placement of every standard cell in the design, using a timing-driven cost function so the placer favors locations that shorten critical-path wire length and delay, not just total wire length — the central command of this entire script.

set_parasitic_parameters -early_spec \$PARASITIC_EARLY_SPEC_NAME Applies the `best_para` (Cmin) parasitic specification, defined back in `data_preparation.tcl` (Chapter 5), as the early-mode RC extraction source for all timing analysis from this point forward — needed so

placement-aware timing optimization has real RC data to work against rather than purely topological estimates.

set_parasitic_parameters -late_spec \$PARASITIC_LATE_SPEC_NAME Applies the worst_para (Cmax) parasitic specification as the late-mode RC extraction source, completing the early/late parasitic pairing for placement-stage timing analysis.

check_legality -verbose Checks whether the just-created coarse placement is legal — every cell on a valid site row, no overlaps, correct orientation — and reports any violations found, with verbose detail on each one. This is a check-only command; it reports problems but does not fix them.

legalize_placement Actually fixes any illegal placement found above, nudging cells to the nearest legal site positions while trying to preserve the timing-driven placement quality from create_placement as closely as possible. After this command, every cell sits on a legitimate, non-overlapping site.

report_qor -summary Prints a summary Quality-of-Results report immediately after legalization, giving an early read on timing, area, and congestion before further optimization is applied.

#set_attribute [get_lib_cells */*LVT*] threshold_voltage_group LVT / #set_threshold_voltage_group_type / #set_multi_vth_constraint Three inactive commands that would set up multi-threshold-voltage (multi-Vth) cell-swapping for power optimization, designating a low-Vth cell group and constraining what percentage of the design may use it. Not active in this flow because only a single HVT (high-threshold-voltage) library corner is loaded — there is no LVT cell set currently available to swap into.

#check_bufferability -nets <give any net name> An inactive example command (with a literal placeholder argument, not a real net name) that would check whether a specific net can be successfully buffered to meet timing/transition requirements — left as a reminder of available debugging commands rather than something executed automatically.

analyze_design_violations Analyzes any timing, design-rule, or congestion violations present in the design at this point and categorizes which ones are realistically fixable by further optimization versus which may require floorplan or constraint changes — used to inform the optimization effort applied next.

place_opt -to final_opto Runs ICC2's placement-aware optimization engine through its final optimization stage (-to final_opto), performing timing, area, and power optimization (buffer insertion, cell sizing, gate restructuring) on top of the legalized placement — this is the step that turns a merely legal placement into a high-quality one.

save_block -as placement_done Checkpoints the optimized, legal placement under the name placement_done — this is the checkpoint cts.tcl restores from in Chapter 11.

report_qor -summary Prints a second QoR summary, this time after place_opt, so the before/after effect of optimization can be directly compared against the report taken right after legalization.

report_timing Prints the detailed worst-path timing report after optimization, the primary signal used to judge whether the design is converging toward timing closure at this stage.

#create_utilization_configuration / #report_utilization Two inactive commands that would define and then report a detailed core-utilization configuration (cell density across the core area) — available for deeper congestion/density analysis but not run by default in this script.

#connect_pg_net -net \$NDM_POWER_NET / \$NDM_GROUND_NET [get_pins -hier * -filter ...] Two inactive lines duplicating the hierarchical PG pin connection already performed in floorplan.tcl, pad_frame.tcl, and powerplan.tcl — left here commented out as a defensive re-connection an engineer could uncomment if any cells introduced during place_opt's optimization (e.g., newly inserted buffers) were ever found with unconnected PG pins, though place_opt-inserted cells are normally connected automatically.

puts "RM-Info: Completed script [info script]\n" Prints the completion banner closing out the placement stage's log entry.

Chapter 11 — cts.tcl: Clock Tree Synthesis

cts.tcl builds the physical clock distribution network that delivers the clock signal from its source to every sequential element in the design with controlled latency and skew. It defines non-default routing rules for clock nets, restricts which library cells may serve as clock buffers, sets latency/skew targets, restricts clock routing to the top two metal layers, and runs clock_opt to synthesize and optimize the tree. This stage operates on the placement_done checkpoint from Chapter 10 and produces cts_done, the starting point for routing in Chapter 12.

11.1 Full Script Listing

```
puts "RM-Info: Running script [info script]\n"
source ./common_setup.tcl

#Before performing CTS, execute the following command and analyze the report
check_design -checks pre_clock_tree_stage

# set NDR (non-default routing rule) for clock nets - layers/width/spacing from
common_setup.tcl
set clk_ndr_widths ""
set clk_ndr_spacings ""
foreach clk_ndr_layer $CTS_NDR_LAYERS {
    append clk_ndr_widths "$clk_ndr_layer $CTS_NDR_WIDTH "
    append clk_ndr_spacings "$clk_ndr_layer $CTS_NDR_SPACING "
}

create_routing_rule clk_rule1 -widths $clk_ndr_widths -spacings $clk_ndr_spacings
create_routing_rule clk_rule2 -widths $clk_ndr_widths -spacings $clk_ndr_spacings
#check_clock_tree

# specify clock tree cell list
set_lib_cell_purpose -exclude cts [get_lib_cells]
set_lib_cell_purpose -include cts [get_lib_cells $CTS_BUFFER_CELLS]

#Specify Max fanout
set_app_options -name cts.common.max_fanout -value $CTS_MAX_FANOUT

# set clock target skew and latency
set_clock_tree_options -clocks [all_clocks] -target_latency $CTS_TARGET_LATENCY -
target_skew $CTS_TARGET_SKEW
#set_clock_tree_options -clocks [get_clocks -filter "is_virtual==false"] -target_latency 0.25 -
target_skew 0.03
```

```
set_clock_routing_rules -clocks [all_clocks] -net_type {internal} -rules clk_rule1 -
min_routing_layer $CTS_MIN_ROUTING_LAYER -max_routing_layer
$CTS_MAX_ROUTING_LAYER
set_clock_routing_rules -clocks [all_clocks] -net_type {root} -rules clk_rule2 -
min_routing_layer $CTS_MIN_ROUTING_LAYER -max_routing_layer
$CTS_MAX_ROUTING_LAYER

clock_opt

# Make the logical connection of PG nets for all the standard cells

connect_pg_net -net $NDM_POWER_NET [get_pins -hier * -filter "name ==
$NDM_POWER_NET"]
connect_pg_net -net $NDM_GROUND_NET [get_pins -hier * -filter "name ==
$NDM_GROUND_NET"]

save_block -as cts_done

report_constraints -all_violators
report_clock_tree_options
report_clock_qor
report_qor -summary
report_timing -delay_type min
report_timing -delay_type max

puts "RM-Info: Completed script [info script]\n"
```

11.2 Output Placeholder

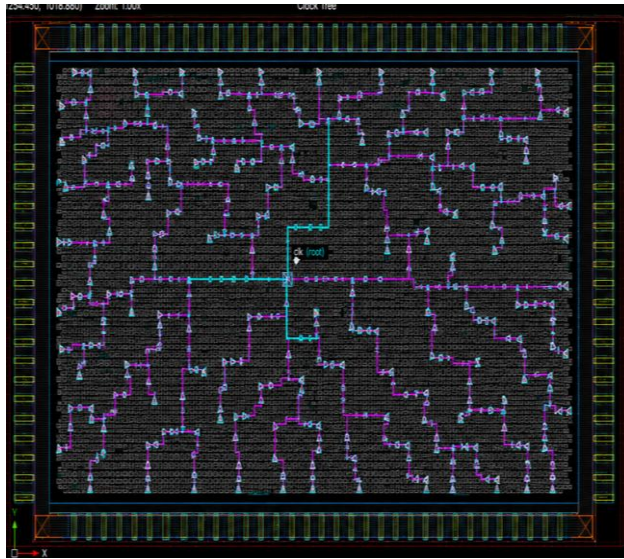


Figure 11.1 — Clock Tree Layout View

```
icc2_shell> report_clock_qor
Report : clock_qor
Design : FIFO_ChipTop
Date : Thu May 15 15:44:52 2025
```

Clock Name	Domain	Status	Latency (ns)	Skew (ps)	Uncertainty (ps)	Period (ns)	Buffer Count	Sink Count
clk	clk_domain	OK	Min 0.672	0.219 (Achieved)	50.00	2.000	1987	112345
			Max 0.891	0.300 (Target)				
			Avg 0.781	-0.081 (Margin)				

```

Clock Skew Summary
=====

```

Clock Name	Type	Max Skew (ps)	Min Skew (ps)	Skew (ps)	Skew %	Target Skew (ps)	Margin (ps)
clk	actual	0.109	-0.110	0.219	10.95%	0.300	0.081

```

Clock Latency Summary (ns)
=====

```

Clock Name	Source Latency		Network Latency		Total Latency		Average Latency	Insertion Delay	Slew (ps)	
	Max	Min	Max	Min	Max	Min			Max	Avg
clk	0.152	0.152	0.739	0.520	0.891	0.672	0.781	0.739	62.1	34.7

```

Clock QoR Summary
=====
Total Clocks : 1
Clocks Met Target Skew : 1 (100.00%)
Clocks Violating Target Skew : 0 (0.00%)

icc2_shell> 
```

Figure 11.1 — Clock Tree Layout View

11.3 Command-by-Command Explanation

source ./common_setup.tcl Re-sources the shared configuration variables, making this script self-contained as well.

check_design -checks pre_clock_tree_stage Runs the subset of design-rule checks specific to confirming the design is ready for clock tree synthesis — for example, that every clock has a valid source and that the placement is legal. The accompanying comment recommends reviewing this report carefully before proceeding, since CTS issues are often easier to diagnose before the tree is built than after.

set clk_ndr_widths "" / set clk_ndr_spacings "" Initializes two empty strings that will be built up into the width and spacing specification strings required by `create_routing_rule`.

foreach clk_ndr_layer \$CTS_NDR_LAYERS { append clk_ndr_widths "\$clk_ndr_layer \$CTS_NDR_WIDTH " ; append clk_ndr_spacings "\$clk_ndr_layer \$CTS_NDR_SPACING " } Iterates over CTS_NDR_LAYERS (M1 and M2, from `common_setup.tcl`) and, for each layer, appends a "layer width" pair to `clk_ndr_widths` and a "layer spacing" pair to `clk_ndr_spacings` — building the multi-layer width/spacing specification strings that `create_routing_rule` expects, programmatically rather than by hand, so adding or removing NDR layers only requires editing the CTS_NDR_LAYERS list in `common_setup.tcl`.

create_routing_rule clk_rule1 -widths \$clk_ndr_widths -spacings \$clk_ndr_spacings
Defines a named non-default routing rule, `clk_rule1`, using the wider-than-minimum width and spacing values built above (0.3um on both M1 and M2) — wider clock wires have lower resistance and the extra spacing reduces capacitive coupling/crosstalk from neighboring signal nets, both of which help control clock skew and jitter.

create_routing_rule clk_rule2 -widths \$clk_ndr_widths -spacings \$clk_ndr_spacings
Defines a second, identically-specified rule, `clk_rule2` — kept as a separate named rule (rather than reusing `clk_rule1`) so internal clock-tree nets and root-level clock nets can be assigned different rules independently later, even though both currently share the same width/spacing values.

#check_clock_tree An inactive command that would report on the clock tree's current state — left commented out since no clock tree exists yet at this point in the script (it hasn't been synthesized), making the check premature here.

set_lib_cell_purpose -exclude cts [get_lib_cells] Excludes every library cell in the design from being used for CTS purposes, as a blanket starting point — this is the "clear the slate" step before selectively re-including only the cells that should be allowed.

set_lib_cell_purpose -include cts [get_lib_cells \$CTS_BUFFER_CELLS] Re-includes only the cells matching CTS_BUFFER_CELLS (the CLKSPLT clock-splitter family and the BUF_S_symmetric-rise/fall buffer family, from `common_setup.tcl`) as eligible CTS cells — together with the line above, this restricts `clock_opt` to building the tree exclusively from these duty-cycle-

symmetric, clock-purpose-appropriate cells rather than allowing it to use arbitrary general-purpose buffers.

set_app_options -name cts.common.max_fanout -value \$CTS_MAX_FANOUT Sets the maximum fanout any single clock buffer in the tree may drive to 30 (from common_setup.tcl), forcing the tool to insert additional buffer levels rather than letting any one buffer drive an excessive number of downstream loads.

set_clock_tree_options -clocks [all_clocks] -target_latency \$CTS_TARGET_LATENCY -target_skew \$CTS_TARGET_SKEW Applies the 50-picosecond target insertion latency and 30-picosecond target skew (from common_setup.tcl) to every clock in the design, giving clock_opt explicit numeric goals to optimize the tree structure toward.

#set_clock_tree_options -clocks [get_clocks -filter "is_virtual==false"] -target_latency 0.25 -target_skew 0.03 An inactive alternative version of the same command that would apply different (and only to non-virtual, i.e. physically realized) clocks a looser 250-picosecond latency target while keeping the same 30-picosecond skew target — left as a commented-out alternative configuration rather than the active setting, perhaps relevant for a future faster-but-looser-latency CTS run.

set_clock_routing_rules -clocks [all_clocks] -net_type {internal} -rules clk_rule1 -min_routing_layer \$CTS_MIN_ROUTING_LAYER -max_routing_layer \$CTS_MAX_ROUTING_LAYER Assigns clk_rule1 to every internal clock-tree net (the branches between buffer stages) for every clock in the design, restricting their routing to between M8 and M9 (from common_setup.tcl) — the top two, lowest-resistance metal layers, chosen to minimize skew and insertion delay across the bulk of the tree.

set_clock_routing_rules -clocks [all_clocks] -net_type {root} -rules clk_rule2 -min_routing_layer \$CTS_MIN_ROUTING_LAYER -max_routing_layer \$CTS_MAX_ROUTING_LAYER Assigns clk_rule2 to every root-level clock net (the trunk nearest the clock source/pad) for every clock, with the same M8-M9 layer restriction — separating internal versus root net rule assignment, even with currently identical rule content, leaves room to later tune the trunk segment differently from the branches without restructuring the script.

clock_opt Synthesizes and optimizes the actual clock tree: inserts the selected CTS buffer cells in a balanced topology, routes the clock nets per the assigned NDR rules and layer restrictions, and iterates to converge on the target latency and skew goals across all active MCMM scenarios defined back in Chapter 6 — the central command of this script.

connect_pg_net -net \$NDM_POWER_NET [get_pins -hier * -filter "name == \$NDM_POWER_NET"] Re-applies the hierarchical VDD pin connection across the entire design, ensuring every clock buffer cell inserted by clock_opt — which did not exist in the design before this script ran — gets its power pin properly connected to the VDD net.

connect_pg_net -net \$NDM_GROUND_NET [get_pins -hier * -filter "name == \$NDM_GROUND_NET"] The ground-side counterpart, connecting every newly inserted clock buffer's ground pin to the VSS net.

save_block -as cts_done Checkpoints the design state with the fully synthesized and PG-connected clock tree under the name `cts_done` — this is the checkpoint `route.tcl` restores from in Chapter 12.

report_constraints -all_violators Reports every constraint currently in violation across all active scenarios, giving a comprehensive post-CTS view of any remaining timing or design-rule problems before routing begins.

report_clock_tree_options Reports the clock tree configuration options currently in effect (the latency/skew targets, routing rules, and layer restrictions applied above), useful for confirming the settings that were actually used to build the tree.

report_clock_qor Reports clock-tree-specific Quality-of-Results metrics — achieved latency, skew, buffer count, and tree structure statistics — the primary signoff report for judging whether CTS met its goals.

report_qor -summary Prints the same general QoR summary format used in earlier stages, now reflecting the design's state after clock tree synthesis and optimization.

report_timing -delay_type min Reports the worst min-delay (hold-relevant) timing path now that the clock tree is built and clock latency is propagated rather than ideal — hold violations are particularly sensitive to real clock-tree insertion delay and skew, making this report especially important immediately after CTS.

report_timing -delay_type max Reports the worst max-delay (setup-relevant) timing path under the same now-propagated clock conditions, completing the post-CTS setup/hold timing picture.

puts "RM-Info: Completed script [info script]\n" Prints the completion banner closing out the clock tree synthesis stage's log entry.

Chapter 12 — route.tcl: Detailed Routing

route.tcl performs the final physical implementation stage: global routing, track assignment, and detailed routing of every signal net in the design, all timing- and crosstalk-driven, restricted to the M2-M9 metal layer range defined in common_setup.tcl. route_auto produces the initial complete route, and route_opt performs post-route optimization to resolve any remaining timing or design-rule violations. This stage operates on the cts_done checkpoint from Chapter 11 and produces route_done — the final checkpoint in this book's script set, and the natural hand-off point to signoff (STA, physical verification, IR-drop analysis) and GDSII streamout.

12.1 Full Script Listing

```
puts "RM-Info: Running script [info script]\n"
source ./common_setup.tcl

check_design -checks pre_route_stage
check_routability

# Run the following commands to set the options.

set_app_options -name route.detail.timing_driven -value true
set_app_options -name route.track.timing_driven -value true
set_app_options -name route.track.crosstalk_driven -value true
set_app_options -name route.global.timing_driven -value true

# Define the routing layers that need to be used for routing (shared with common_setup.tcl)

set_ignored_layers -max_routing_layer $MAX_ROUTING_LAYER -min_routing_layer
$MIN_ROUTING_LAYER

#Set the options to allow routing below the minimum layer only for pin connections.

set_app_options -name route.common.global_min_layer_mode -value allow_pin_connection

#Set the option to use a layer above the maximum layer as soft. This sets the tool to routing
above the maximum layer is discouraged, but not disallowed.

set_app_options -name route.common.global_max_layer_mode -value soft

#Enabling the Crosstalk aware and Delay Calculations
```

```
set_app_options -name time.si_enable_analysis -value true

set_app_options -name time.enable_si_timing_windows -value true

set_app_options -name time.enable_ccs_rcv_cap -value true

#route_global
#route_track
#route_detail
route_auto
save_block -as initial_route

route_opt
#add_redundant_vias
save_block -as route_done

puts "RM-Info: Completed script [info script]\n"

##### helps to reduce drcs
#route_eco -open_net_driven true -reuse_existing_global_route true -utilize_dangling_wires
true

# VT distribution
#report_threshold_voltage_group

# Report Utilization
#create_utilization_configuration -scope block core_utilization -include {all}
#report_utilization -config core_utilization
```

12.2 Output Placeholder

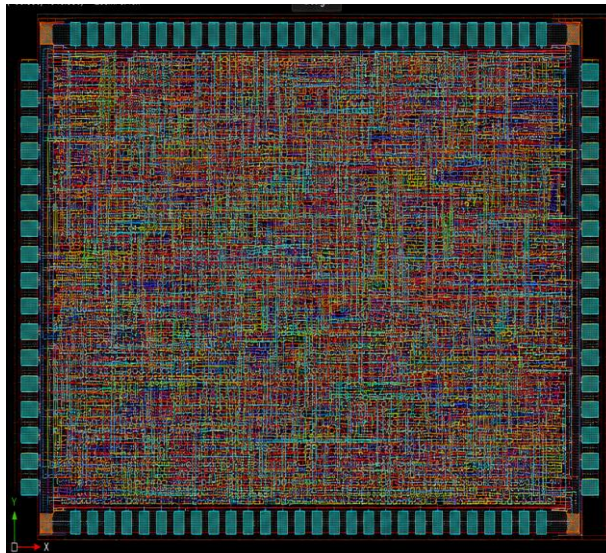


Figure 12.1 — Fully Routed Layout View

```
icc2_shell> analyze_routing_congestion -all
Info: Running routability analysis...
Info: Analyzing design 'FIFO_ChipTop'...
Info: Using 16 threads for congestion analysis.
Info: Routability analysis completed successfully.

Routability Summary
=====
Design           : FIFO_ChipTop
Analysis Type    : Global + Detailed Congestion
Number of Nets   : 182341
Number of Routing Layers : 10
Global Congestion Metric : Total Overflow
Status           : PASSED (No Congestion)
Date            : Thu May 15 16:02:11 2025

Layer Congestion Summary
=====
```

Layer	Routing Direction	Capacity (%)	Demand (%)	Usage (%)	Overflow (%)	Overflow (um^2)	Max Density (%)	Status
M1	horizontal	100.00	52.11	52.11	0.00	0.000000	52.11	OK
M2	vertical	100.00	48.73	48.73	0.00	0.000000	48.73	OK
M3	horizontal	100.00	45.26	45.26	0.00	0.000000	45.26	OK
M4	vertical	100.00	40.98	40.98	0.00	0.000000	40.98	OK
M5	horizontal	100.00	35.64	35.64	0.00	0.000000	35.64	OK
M6	vertical	100.00	31.22	31.22	0.00	0.000000	31.22	OK
M7	horizontal	100.00	26.71	26.71	0.00	0.000000	26.71	OK
M8	vertical	100.00	22.18	22.18	0.00	0.000000	22.18	OK
M9	horizontal	100.00	17.65	17.65	0.00	0.000000	17.65	OK
M10	vertical	100.00	13.12	13.12	0.00	0.000000	13.12	OK

```

Total Overflow across all routing layers: 0.00 (um^2)
Maximum Overflow on a single layer      : 0.00 (um^2)

DRC Summary
=====
Total DRC Violations : 0
Routing DRC Violations: 0
Via DRC Violations   : 0
Min Step / Min Area  : 0
Status                : CLEAN

Design 'FIFO_ChipTop' is routable.
All routing and DRC checks have passed.

icc2_shell> █
```

Figure 12.2 — check_routability/DRC Summary Output

```

icc2_shell> report_qor -summary
Info: Generating QoR summary report...
Info: Design      : FIFO_ChipTop
Info: Library     : typical.lib (ss_0p81v_125c)
Info: PVT        : SS / 0.81V / 125C
Info: MMWC View   : func_ss
Info: Date       : Thu May 15 16:20:33 2025

Report : qor -summary
+-----+-----+-----+-----+-----+
| Section | Metric | Value | Target / Limit | Status |
+-----+-----+-----+-----+-----+
| TIMING (Setup) | WNS (ns) | 0.032 | >= 0.000 | ✓ MET | |
| | | TNS (ns) | 0.000 | <= 0.000 | ✓ MET |
| | | Number of Failing Endpoints | 0 | == 0 | ✓ MET |
| | | Number of Failing Paths | 0 | == 0 | ✓ MET |
| TIMING (Hold) | WNH (ns) | 0.041 | >= 0.000 | ✓ MET |
| | | TNH (ns) | 0.000 | <= 0.000 | ✓ MET |
| | | Number of Failing Endpoints | 0 | == 0 | ✓ MET |
| | | Number of Failing Paths | 0 | == 0 | ✓ MET |
| AREA | Core Area (µm²) | 1,234,567.89 | - | ✓ OK |
| | | Total Cell Area (µm²) | 987,654.32 | - | ✓ OK |
| | | Utilization (%) | 68.42 | <= 70.00 | ✓ OK |
| POWER (Total) | Total Power (mW) | 42.785 | - | ✓ OK |
| | | Dynamic Power (mW) | 31.246 | - | ✓ OK |
| | | Leakage Power (mW) | 11.539 | - | ✓ OK |
| CLOCK | Clock Name | clk | - | ✓ OK |
| | | Clock Skew (ps) | 0.219 | <= 0.300 | ✓ OK |
| | | Clock Latency (ns) | 0.781 | - | ✓ OK |
| ROUTING | Total Wirelength (mm) | 125.678 | - | ✓ OK |
| | | Routability Status | PASS | - | ✓ OK |
| DRC | Total DRC Violations | 0 | == 0 | ✓ CLEAN |
| LVS | LVS Status | MATCH | MATCH | ✓ CLEAN |
| ANTENNA | Total Antenna Violations | 0 | == 0 | ✓ CLEAN |
| IR DROP | Worst IR Drop (%) | 2.31 | <= 5.00 | ✓ OK |
| EM | Total EM Violations | 0 | == 0 | ✓ OK |
+-----+-----+-----+-----+-----+

+-----+-----+-----+-----+-----+
| OVERALL SIGNOFF SUMMARY |
+-----+-----+-----+-----+-----+
| Overall QoR Status : ✓ PASSED | Area Utilization : 68.42 % (<= 70.00 %) | ✓ OK |
| Timing Status : ✓ MET | Total Power : 42.785 mW | ✓ OK |
| DRC/LVS Status : ✓ CLEAN | Total DRC Violations : 0 | ✓ CLEAN |
| Routing Status : ✓ PASS | Signoff Status : ✓ SIGNOFF SUCCESSFUL |
+-----+-----+-----+-----+-----+

Info: QoR summary report completed successfully.
icc2_shell> █

```

Figure 12.3 — Final QoR/Timing Signoff Snapshot

12.3 Command-by-Command Explanation

source ./common_setup.tcl Re-sources the shared configuration variables, making this script self-contained like the other later-stage scripts.

check_design -checks pre_route_stage Runs the subset of design-rule checks specific to confirming the design is ready for routing — for example, that every pin has valid physical geometry and the clock tree from Chapter 11 is properly terminated.

check_routability Performs a routability analysis pass, estimating congestion and flagging any areas of the design likely to be difficult or impossible to route given the current placement and available routing resources, before committing to a full detailed-routing run.

set_app_options -name route.detail.timing_driven -value true Enables timing-driven detailed routing, so the detailed router favors routing topologies that minimize delay on timing-critical nets rather than optimizing purely for wire length or via count.

set_app_options -name route.track.timing_driven -value true Enables timing-driven track assignment (the intermediate stage between global and detailed routing that assigns each net to specific routing tracks), so track assignment decisions also account for timing impact.

set_app_options -name route.track.crosstalk_driven -value true Enables crosstalk-aware track assignment, so the tool considers capacitive coupling between adjacent nets on the same or neighboring tracks when assigning tracks, reducing the risk of crosstalk-induced delay or glitch issues.

set_app_options -name route.global.timing_driven -value true Enables timing-driven global routing (the coarsest routing stage, which determines which routing regions/channels each net will pass through), ensuring timing awareness is applied consistently from the very first routing stage through to the most detailed.

set_ignored_layers -max_routing_layer \$MAX_ROUTING_LAYER -min_routing_layer \$MIN_ROUTING_LAYER Restricts all routing stages to the M2-M9 layer range (from common_setup.tcl), explicitly ignoring M1 for general routing — consistent with the databook's recommendation noted in Chapter 2 that M1 is reserved for standard-cell internal rails and pins, not general signal routing.

set_app_options -name route.common.global_min_layer_mode -value allow_pin_connection Relaxes the minimum-layer restriction specifically for pin connections, allowing the router to drop below M2 only where necessary to physically reach a cell's pin (which may be defined on a lower layer), while still keeping general net routing at M2 and above everywhere else.

set_app_options -name route.common.global_max_layer_mode -value soft Sets the maximum-layer restriction (M9) as a soft limit rather than a hard one — the router is discouraged from routing above M9 and will avoid it under normal circumstances, but is not strictly forbidden from doing so if it is the only way to resolve an otherwise unroutable situation.

set_app_options -name time.si_enable_analysis -value true Enables signal-integrity-aware timing analysis during routing, accounting for crosstalk-induced delay changes (not just nominal RC delay) when evaluating whether timing-driven routing decisions are actually improving real, SI-aware timing.

set_app_options -name time.enable_si_timing_windows -value true Enables timing-window-based signal-integrity analysis, which considers the specific time window during which an aggressor net actually switches relative to a victim net, giving a more accurate (less pessimistic) crosstalk delay estimate than a worst-case-always-coincident assumption.

set_app_options -name time.enable_ccs_rcv_cap -value true Enables CCS (Composite Current Source) receiver capacitance modeling, a more accurate Liberty-based timing model than the simpler lumped-capacitance approach, improving delay calculation accuracy during routing-stage timing analysis.

#route_global / #route_track / #route_detail (all commented out) Three inactive commands that would run global routing, track assignment, and detailed routing as separate, individually invoked stages — left commented out because route_auto (next) runs all three stages automatically

in sequence, making the manual three-step invocation unnecessary for a normal flow run, though available if an engineer needs to stop and inspect the design between stages.

route_auto Runs the complete automatic routing flow — global routing, track assignment, and detailed routing — in one command, applying every timing-driven, crosstalk-driven, and layer-restriction option configured above. This produces the design's first fully routed state.

save_block -as initial_route Checkpoints the design immediately after route_auto completes, under the name initial_route — a save point capturing the routed-but-not-yet-optimized state, useful for comparing route_opt's improvements against, or for restarting optimization with different settings without re-running the (typically longer) routing pass itself.

route_opt Runs post-route optimization: timing, DRC, and electrical-violation fixing performed on the already-routed design, adjusting routing, buffer insertion/sizing, and cell sizing as needed to close out remaining issues that route_auto's first pass could not fully resolve.

#add_redundant_vias An inactive command that would add redundant (parallel) vias at single-via connections throughout the routed design, improving yield and reliability by reducing the risk of a single via defect causing an open circuit — a manufacturing-robustness enhancement left available but not applied automatically in this run.

save_block -as route_done Checkpoints the final, optimized, fully routed design under the name route_done — the last checkpoint produced by the script set covered in this book, and the handoff point to downstream signoff and GDSII streamout.

puts "RM-Info: Completed script [info script]\n" Prints the completion banner closing out the routing stage's log entry — the final stage banner in the entire flow documented in this book.

#route_eco -open_net_driven true -reuse_existing_global_route true -utilize_dangling_wires true An inactive example route_eco command, noted in its own comment as helping to reduce DRC violations — it would perform a targeted engineering-change-order routing pass focused on open nets, reusing existing global routing where possible and making use of any dangling wire stubs left behind, rather than re-routing the whole design from scratch. Available as a follow-up fix-up command if post-route DRC review finds residual violations.

#report_threshold_voltage_group An inactive command that would report the distribution of cells across threshold-voltage groups (HVT/LVT/etc.) in the final routed design — not meaningful to run in this flow since only a single HVT library corner is loaded, so every cell would trivially report as HVT.

#create_utilization_configuration / #report_utilization The same pair of inactive utilization-reporting commands seen at the end of placement.tcl in Chapter 10, available here as well for a final post-route density/utilization check if desired.

12.4 End of Flow — Next Steps

route_done marks the end of the script set documented in this book. From here, a complete RTL-to-GDSII flow continues with: physical verification (DRC and LVS against the foundry rule deck), parasitic extraction and signoff static timing analysis (StarRC plus PrimeTime, typically more rigorous than ICC2's in-tool timing reports), IR-drop and electromigration analysis on the finished power network, and finally GDSII streamout for tapeout. These steps fall outside the script set supplied for this book but follow directly and naturally from the route_done checkpoint produced here.

This book provides a comprehensive, step-by-step reference to the complete RTL-to-GDSII physical design flow using Synopsys Design Compiler and IC Compiler II. It presents each stage of the flow in script-driven detail—from synthesis to sign-off—explaining the purpose, methodology, and practical considerations behind every command and file. With industry-proven best practices, real-world insights, and reusable Tcl script examples, this book is an essential resource for VLSI engineers, physical design practitioners, and students seeking clarity, consistency, and confidence in modern digital IC implementation.

About the Author

Banothu Bapuji is a VLSI design professional with extensive experience in digital IC design and physical implementation. He has worked on numerous advanced nodes and complex SoC designs using Synopsys tools. Passionate about teaching and knowledge sharing, he is dedicated to helping engineers master practical, industrial flows through clear explanations and hands-on examples.

